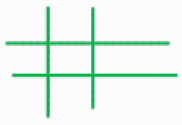
State Space Search :- means start searching from start until we reach the goal while traversing through all states.

State Space → all possible states

↓

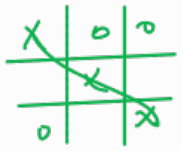
e.g Tic-Tac-Toe



entire board is 1 state

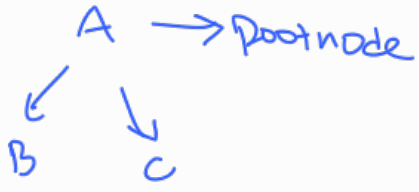
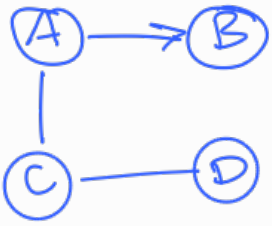
Initial state

making moves →



final state

Path → Sequence of states | Start → Move1 → Move2 → Move3 → Goal

Tree	Graph
→ non linear data str	//
→ collection of nodes and edges	collection of vertices and edges
→ starts with Rootnode	→ No root node (starting node
	
→ No loop	→ loop can be formed
→ Hierarchical model	→ network model

→ Has N-1 edges

Application:
Used to take decision for Searching / deleting / inserting any element in tree

→ It depends on graph

For finding Shortest path in networking

---

optimal solution in AI is the one with lowest cost.

State Space Search → Idea of representing a problem and then solving it.

and that problem is solved via algo's like BFS, DFS, UCS, A*

✡ Note: Use cummulative cost in optimization / Shortest distance UCS / A* algo.

State Space Search have two ways to perform

1) Start from what you know → go foward / Data driven
2) Start from what you want → go backward.

goal driven
e.g Baking a cake

→ Start with initial facts
→ Apply Rules (get new facts)

→ use new facts → apply Rules

→ Reach goal.

Direction of search can affect efficency!

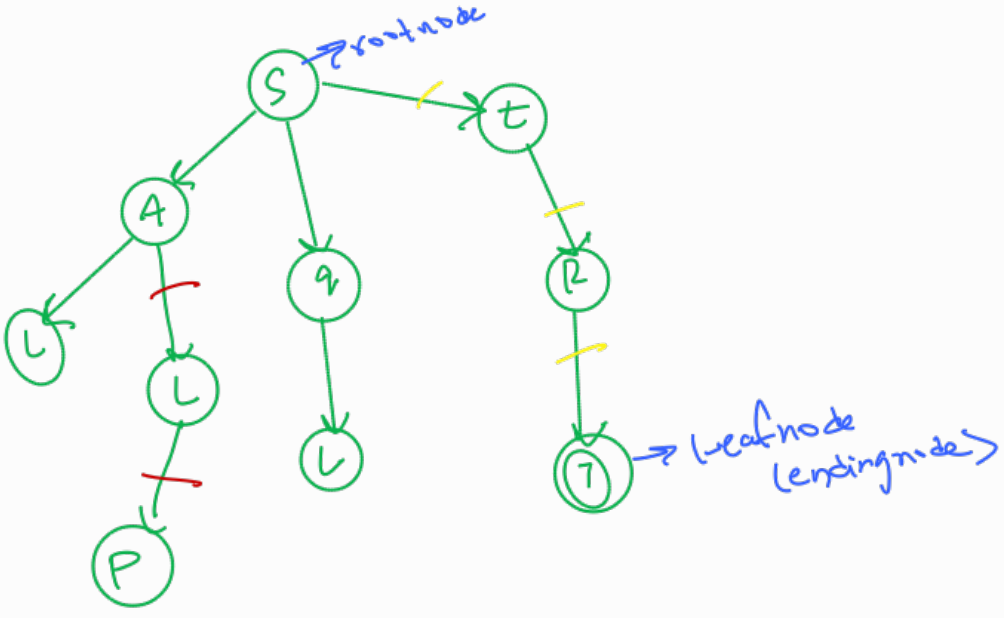
e.g Maze with 1 exit = use Backward as fewer paths

Maze with 1 entrance = use forward Search as fewer path

Branching factor:

height of tree = count edges from leaf node to Root node

=3



Height of Specific node =

e.g A

start from L → A = 2

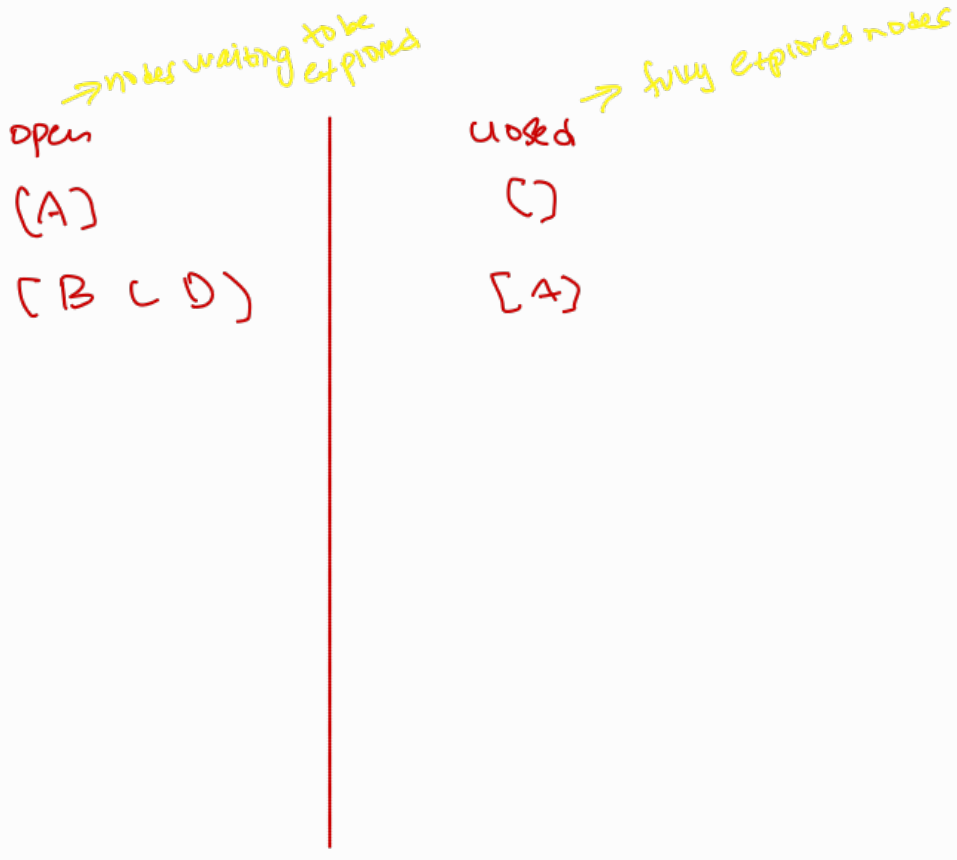
Depth of tree = Root node → Leaf node

Algorithms:

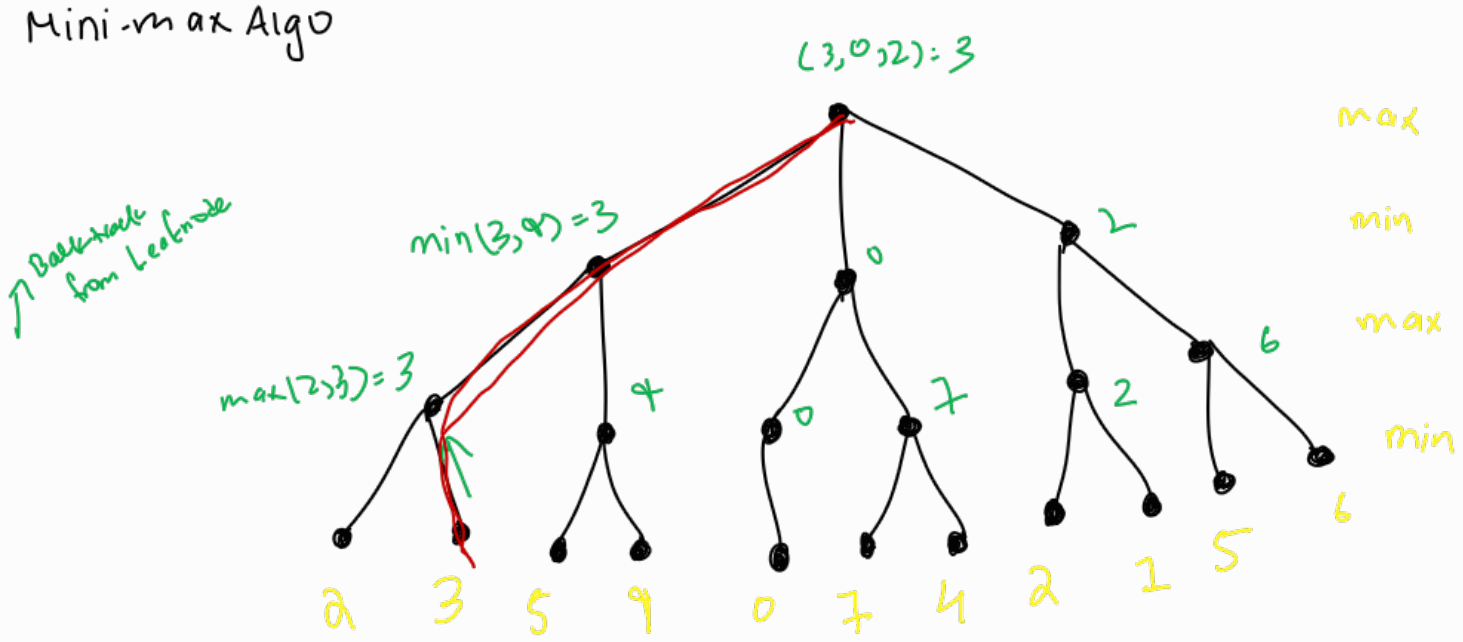
Backtracking Search : try 1 path fully, if it doesn't work go back and try another path.

(not optimal)

(low mem usage)

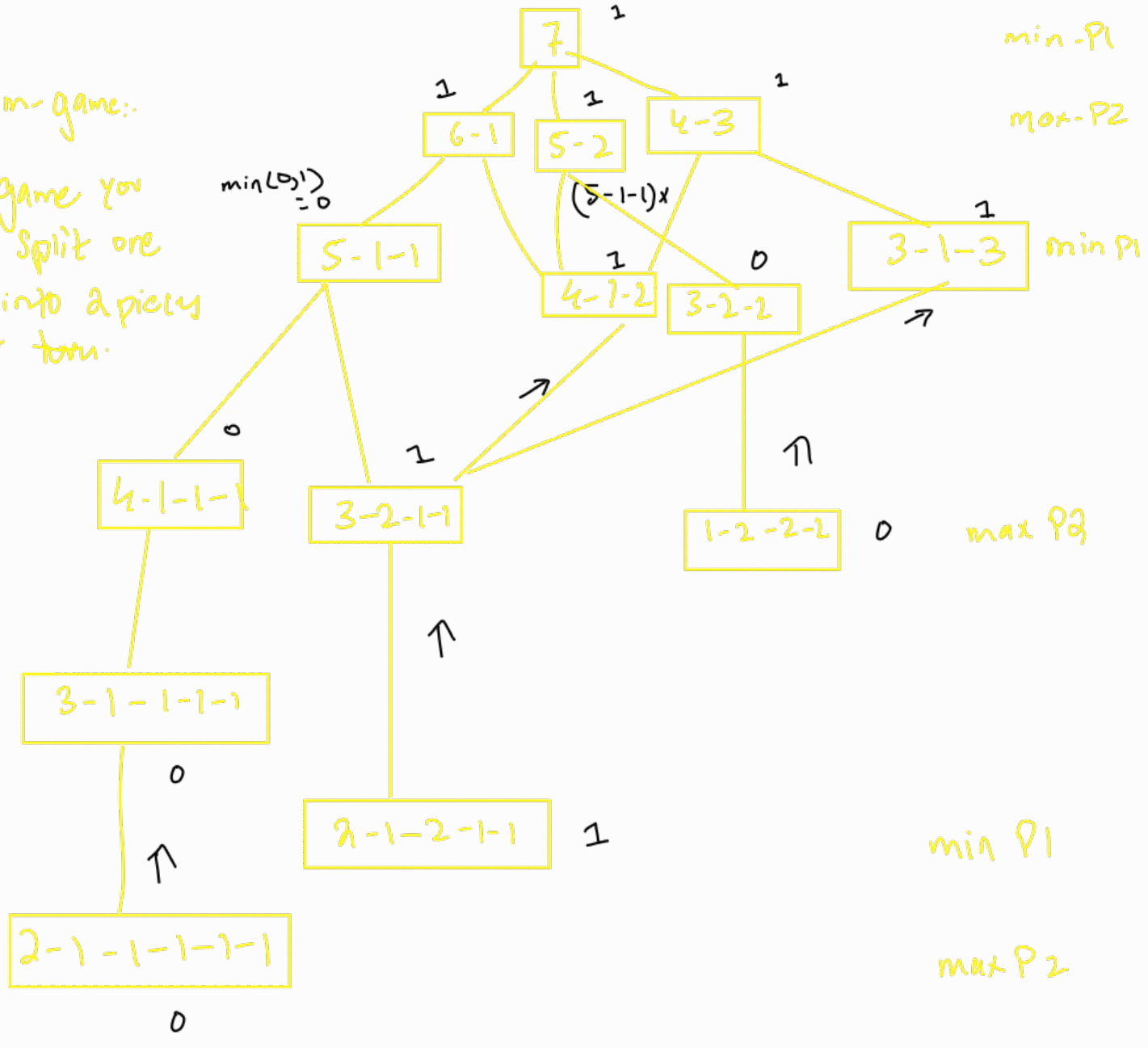


Mini-max Algo



Nim-game:

In this game you can split one pile into a piece per turn.



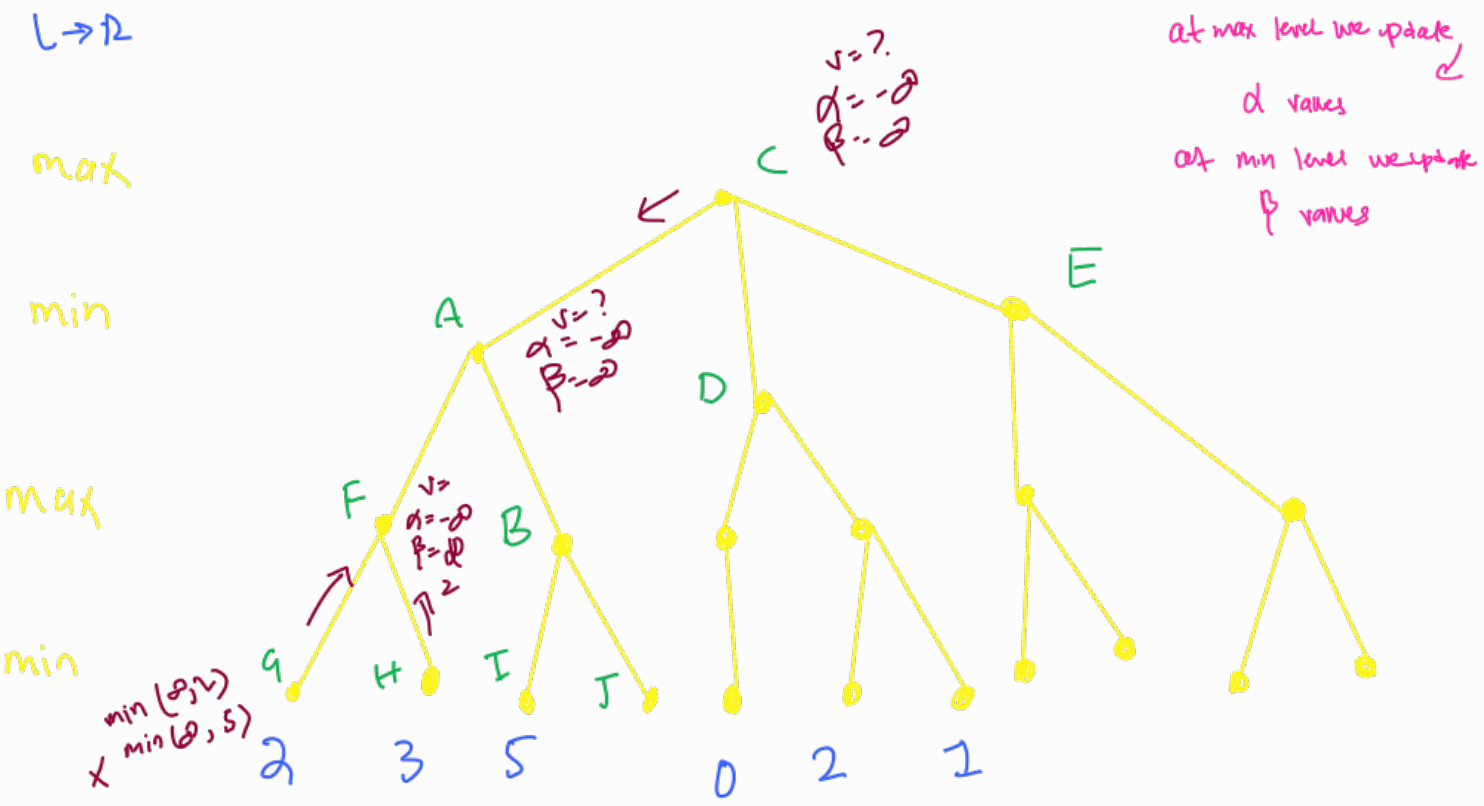
min needs 0 to win.

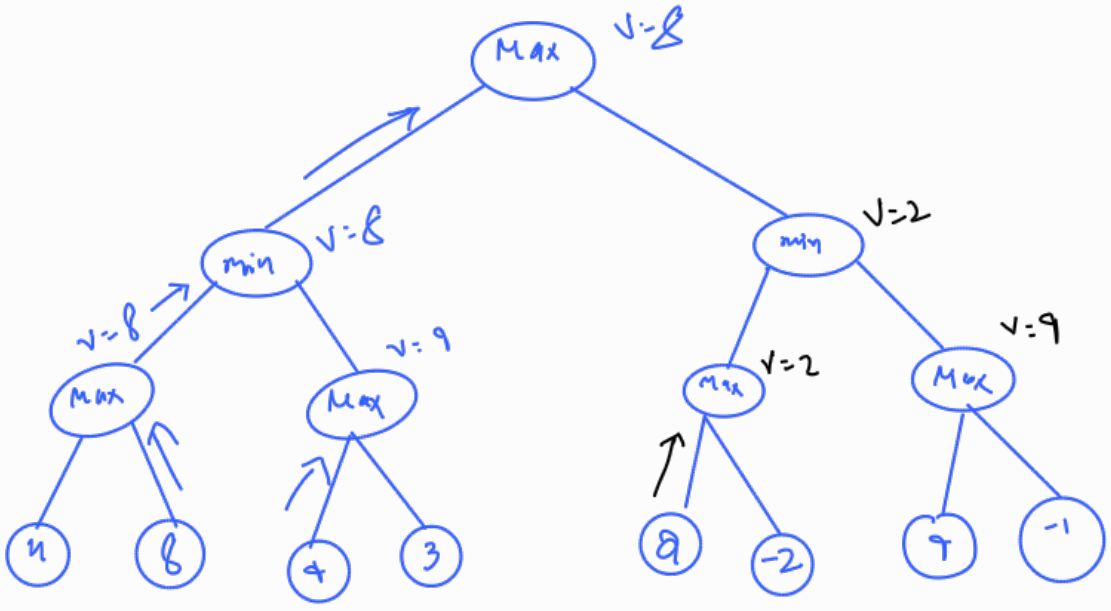
(Start at Leafnodes and backtrack)

if min = 1 it means max won.

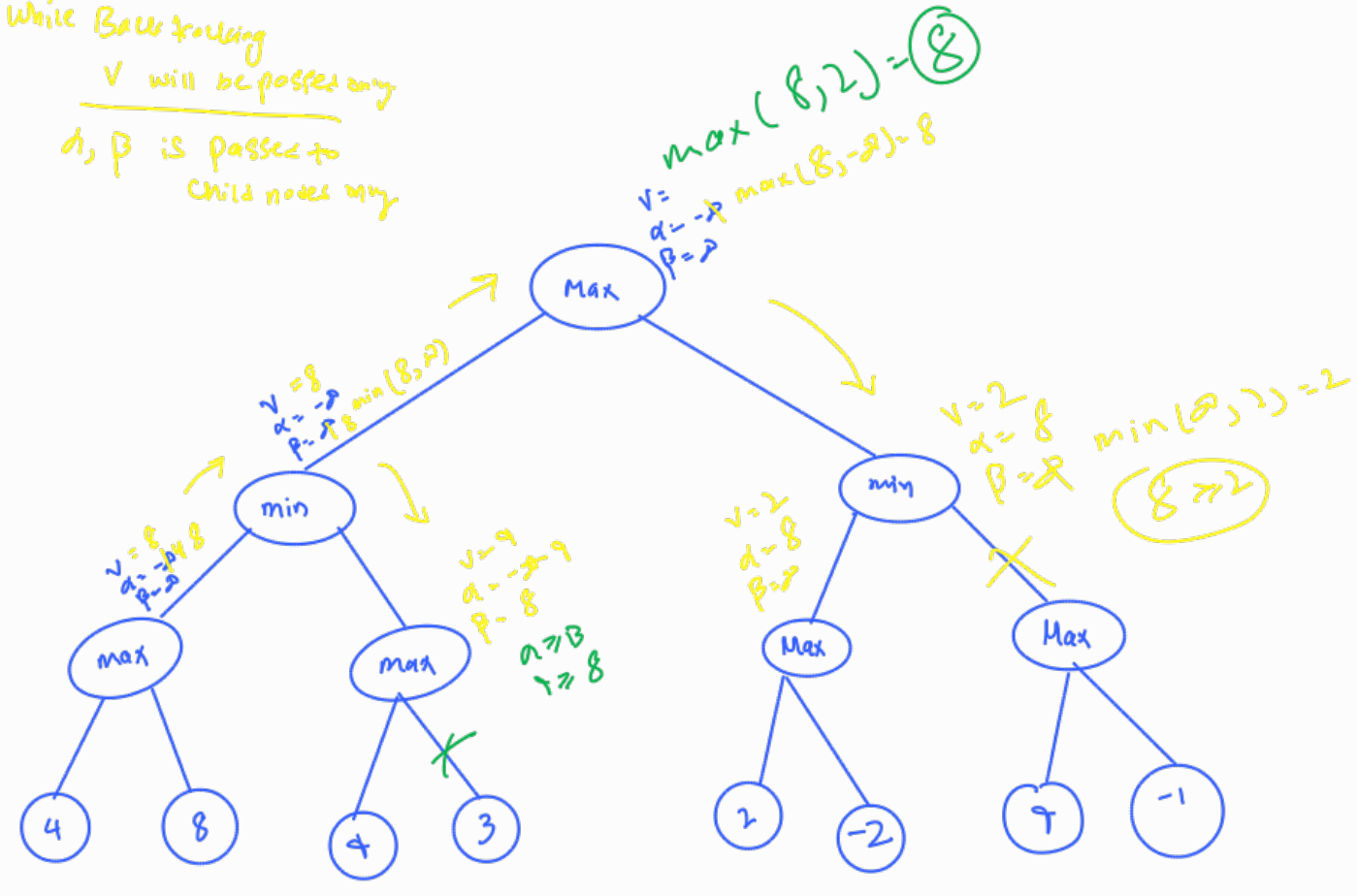
(Alpha-beta pruning algo) → optimized version of mini-max

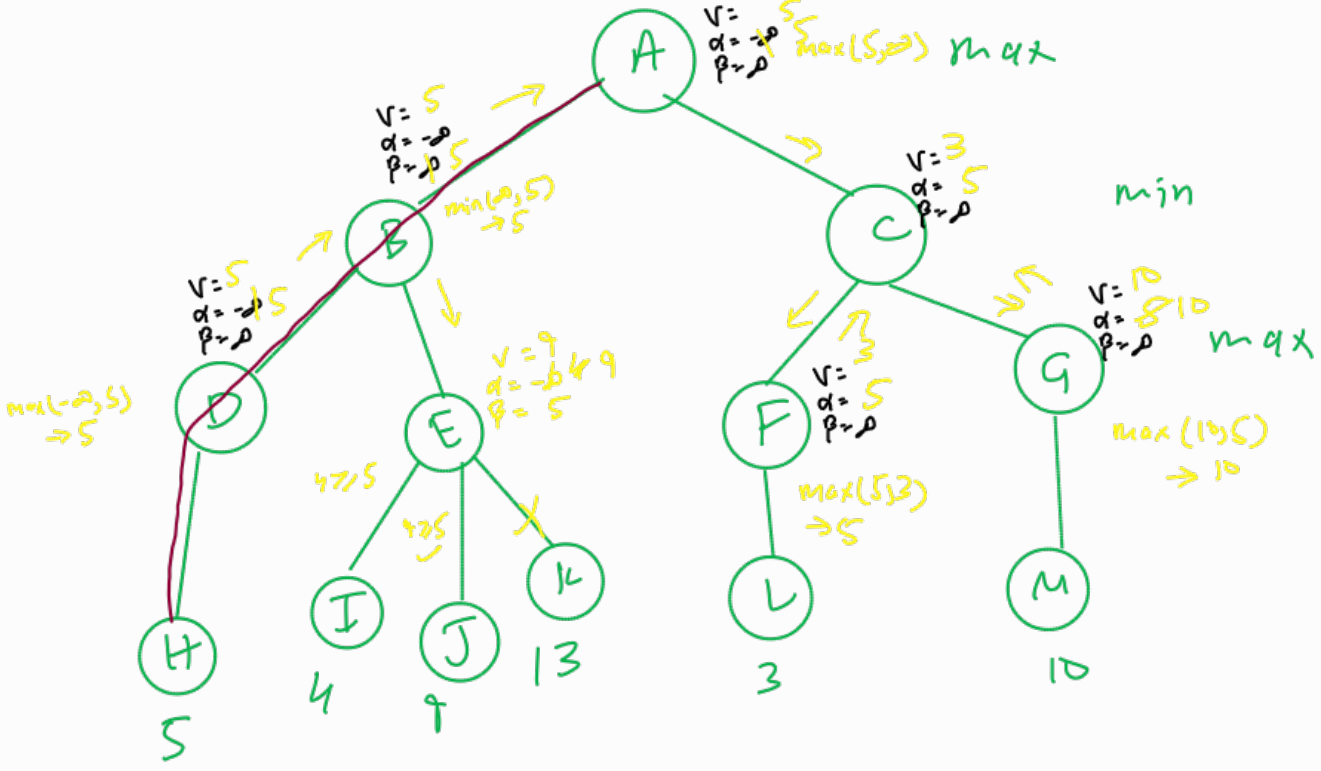
1) update node value
2) update values of α and β $\quad \alpha \leftarrow \max(\alpha, v)$ $\quad \beta \leftarrow \min(\beta, v)$
3) check pruning rule $\alpha \geq \beta$ if this cond not satisfied check the other Branches.





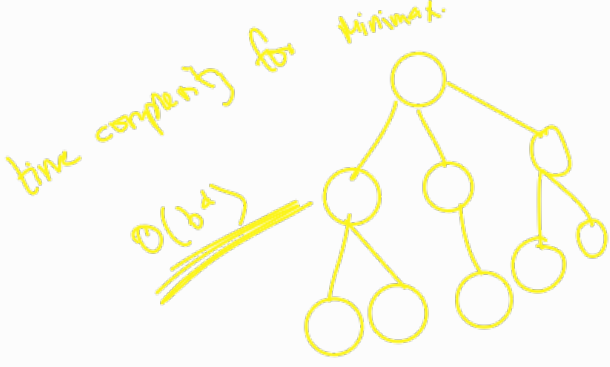
Before Alpha-beta pruning.



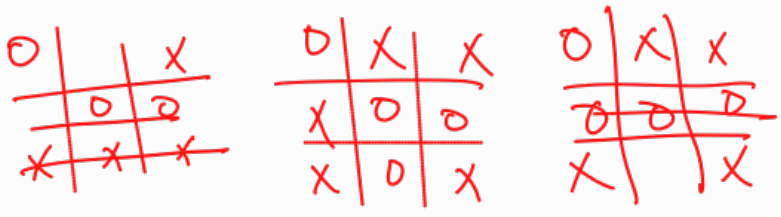


TIC toe minmax.

○ → max X → min



terminal nodes



-1 0 +1

min → 0 min wins

max → 1 max wins

root node : 0 min won

root node = 1 max won

BFS → works likes Queue (FIFO) | Level wise Search.

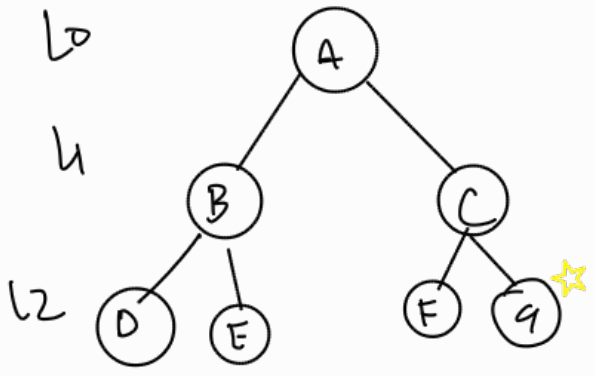
guarantee to give us a sol.

$O(b^d)$ → depth

↓

Branch factor

(How many children every node has)



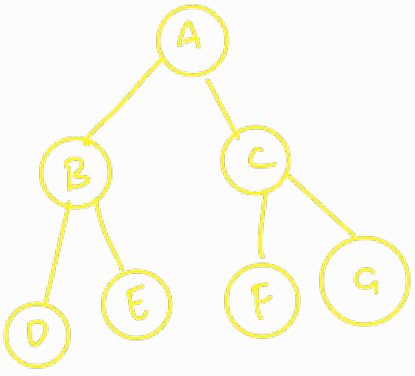
$O(2^2)$

waiting to be checked ← frontier	open	closed	→ Done
	[A]	[]	
expand to childs	[B C]	[A]	
now expand the right most one child at a time	[C D E]	[B A]	
	[D E F G]	[C B A]	
	[E F G]	[D C B A]	
	[F G]	[E D C B A]	
	[G]	[F E D C B A]	
	[]	[G F E D C B A]	

BFS terminates when goal node is removed from open.

Place Children Behind

DFS



Works vertically / Branch wise

Doesn't guarantee a Sol always.

Time comple $O(b^d)$

Place rev items (child at front)

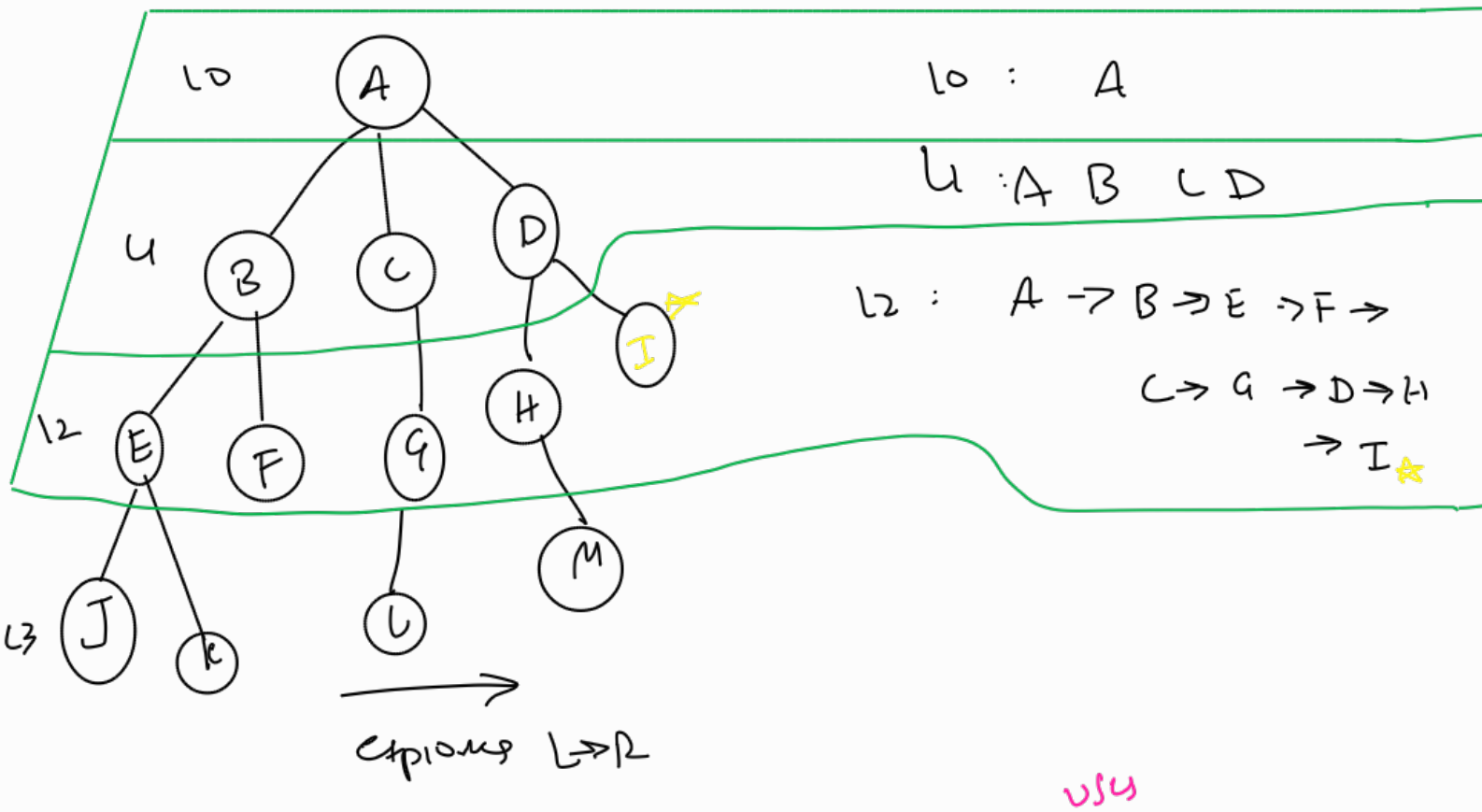
open	closed
(A)	[]
(B C]	(A)
[D E C]	[B A]
[E C]	[D B A]
[C]	[E D B A]
[F G]	[F E D B A]
[G]	[G F [...)
[]	

open	closed
[A]	[ ]
[B C D]	[A]
[E F C D]	[B A]
[K L F C D]	[E B A]
[S L F C D]	[K E B A]
[L F C D]	[S K E B A]
[T F C D]	[L S K E B A]
[F C D]	[T L S K E B A]
[M C D]	[F T L S K E B A]
[C D]	[M . . . .]
[G H D]	[C M . . . .]
⋮	

keep going until we close goal state

Iterative deepening DFS → Searches limit by limit.

combines both DFS + BFS.



Draw Back:
if goal node not found at assigned level, it again restarts from root node and increases the level.

→ Use Stack, push child at front.

IDS

1 S

4 S, A, B, C

8 S, A, D, E, B, C, F, G

10 S, A, D, E, H, I (X) goal found Stop

(✓) , B, C, F, G

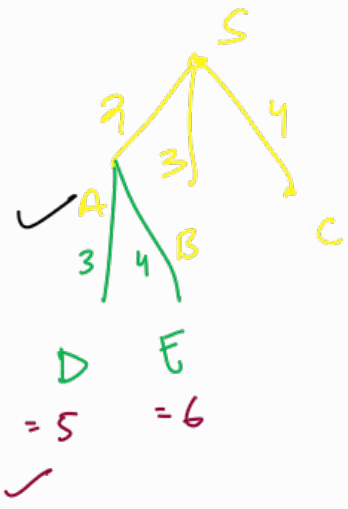
Valid path : S → A → E → I

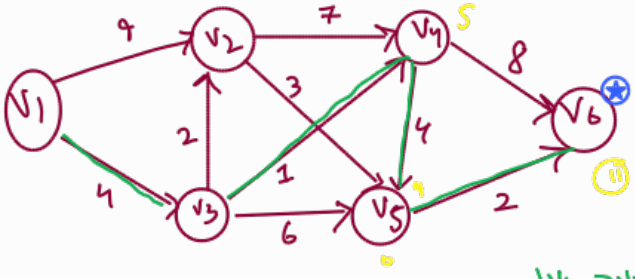
Depth limit search not complete, & are not optimal → if limit is too shallow we might not reach goal → fail → Stack

UCS → (weighted graph) finds the least cost path from start node to goal node

priority queue

→ explores the node with lowest cummulative cost (g(n))

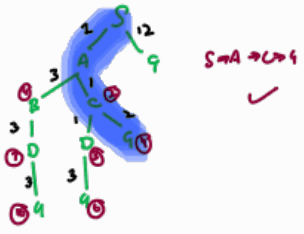




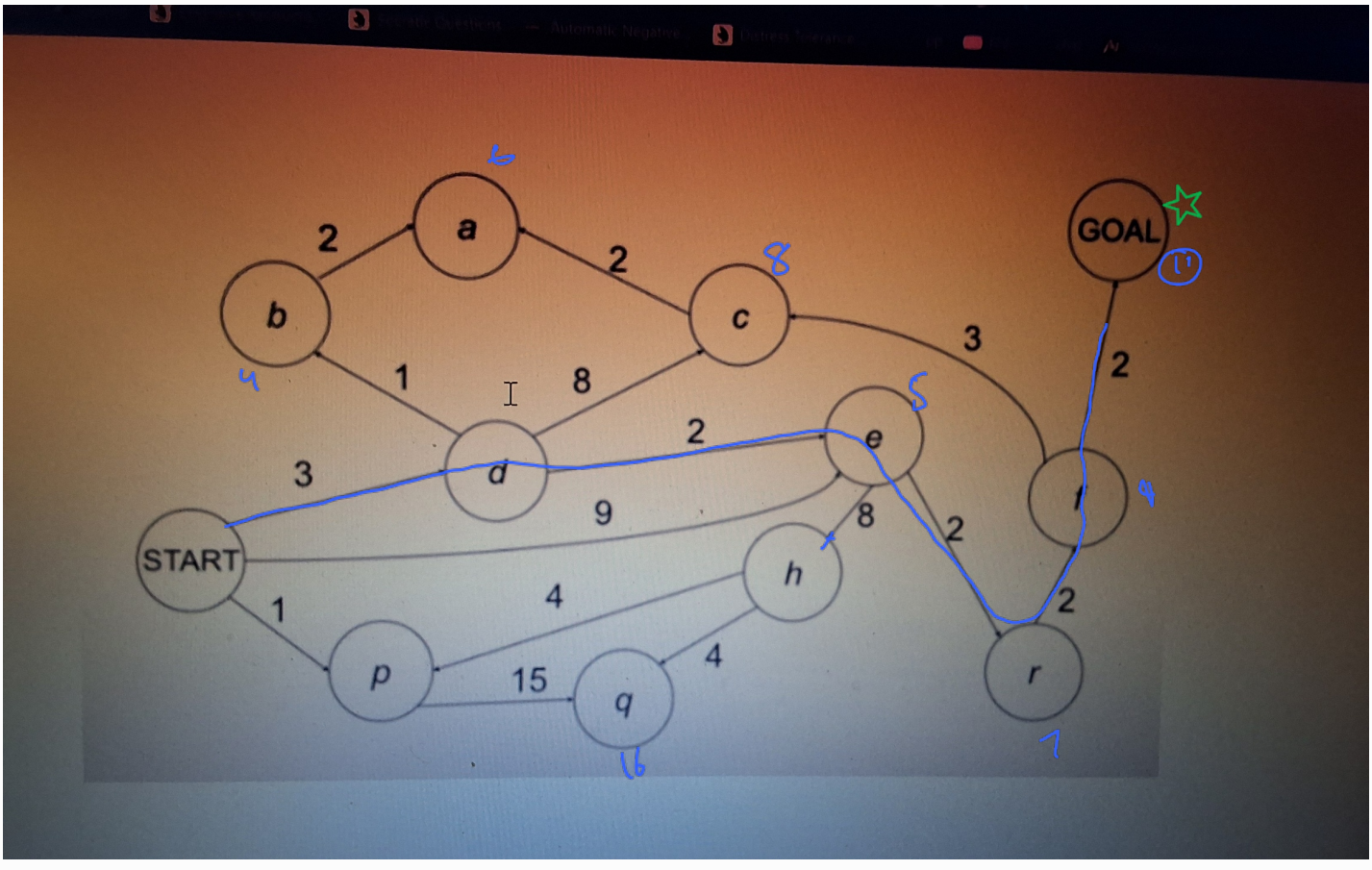
(V1 → V3 → V4 → V5 → V6)

open	closed
V1	
V1	{(V2,9), (V3,4)}
V3	{(V2,9), (V2,6), (V4,5), (V5,10)}
V4	{(V6,13), (V5,9), (V2,9), (V2,6), (V5,10)}
V2	{(V6,13), (V5,9), (V2,9), (V5,10), (V4,9), (V4,13)}
V5	{(V6,11), (V6,13), (V2,9), (V5,10), (V4,13)}
V2	{(V4,16), (V5,12), (V6,11), V6,13, V5,10, V4,13}
V6	

completely optimal.



SMA algorithm



note: path returned by BFS and IDS is same → if all costs are equal.

DFS

open	closed
[S]	[]
[A B]	[S]
[C G B]	[A S]
[G B]	[C* A S]

BFS

open	closed
S	[]
[A B]	[S]
[B C G]	[A S]
[C G G1]	[B A S]
	[C* B A S]

Reason

Path S → A → C

S → A → C

S → L0

S → A → B L1

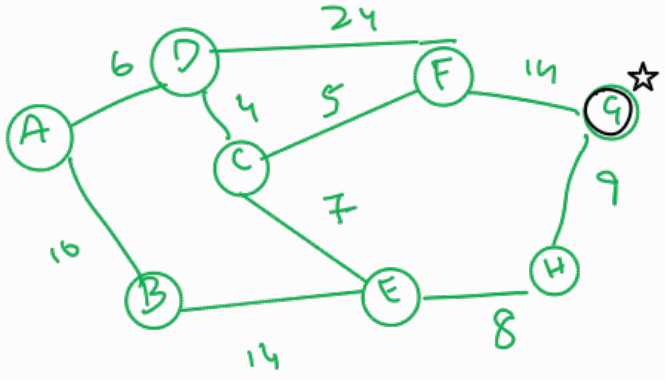
S → A → C* L1

Heuristic (Informed Search) Straight-line distance on a map.

→ Search Space decreases | as heristic tells which node to follow to reach the goal faster.

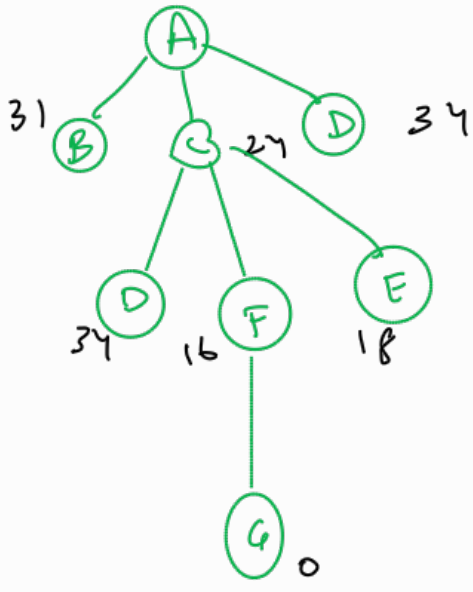
→ Provides an estimated cost b/w current state → goal state

Best first search



node	H(n)
A → G	39
B → G	31
C → G	24
D → G	34
E → G	18
F → G	16
H → G	9
G → G	0

uses this H[n] way

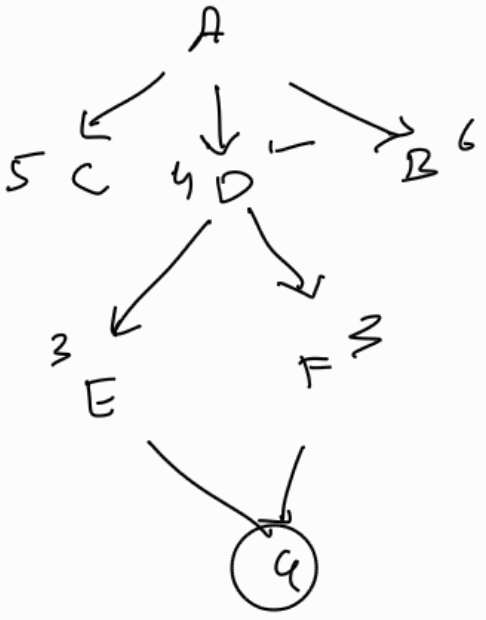


expand the node with least heuristic value

A → C → F → G

e.g greedy Best first Search.

A → G 7
B → G 6
C → G 5
D → G 4
E → G 3
F → G 3
G → G 0



A → D → E → G
or
A → D → F → G (X)

alphabetical order will be followed

Path cost:: 4 + 1 + 4 → (G)

Time complexity:: $O(b^m)$ — b: Branching factor, m: max depth

completeness :: not guaranteed.

optimal :: No

Heuristic
- admissible
- not admissible

h(n) = 0 means close to goal node
h(n) very large means far from goal

(A* algorithm) ←

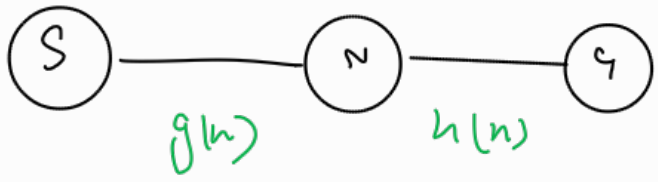
↳ (UCS + greedy BFS)

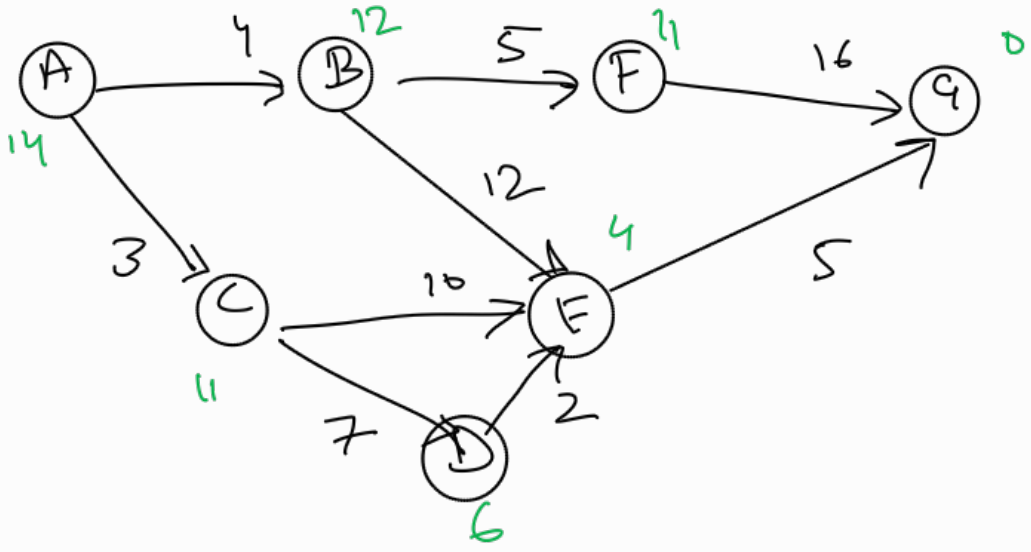
$f(n) = g(n) + h(n)$ → estimated cost from initial node to g.n

↓ actual cost of starting node to reaching that node

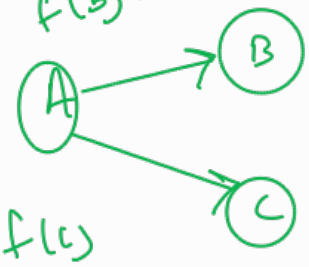
optimal + complete
↓ if heristic admissible

But high mem consup.



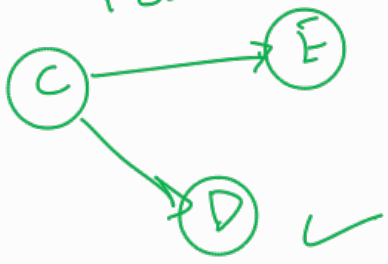


$f(b) = 12+4$

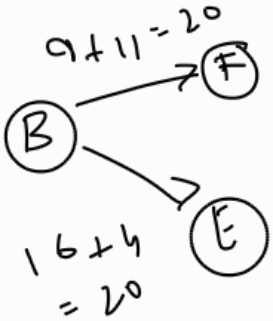


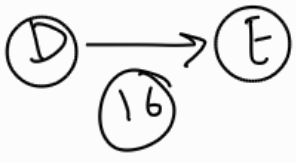
$f(c) = g(c) + h(c) = 3 + 11 = 14$

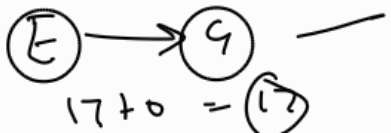
$f(E) = g(E) + h(E) = 13 + 4$



$f(D) = 10 + 6 = 16$







A → C → D → F → G

(SLD)

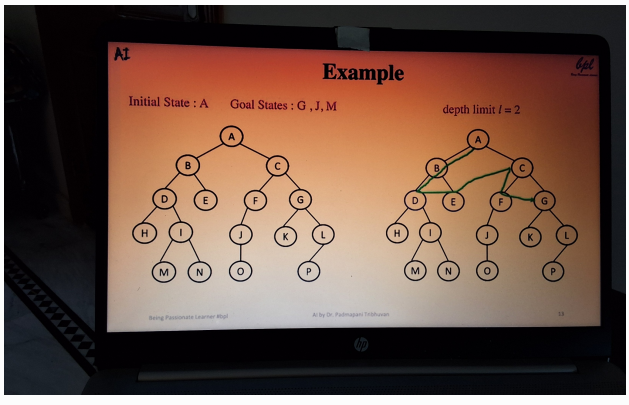
A* algo is complete if heristic value is under-estimated as compared to actual value (g(n))

↓

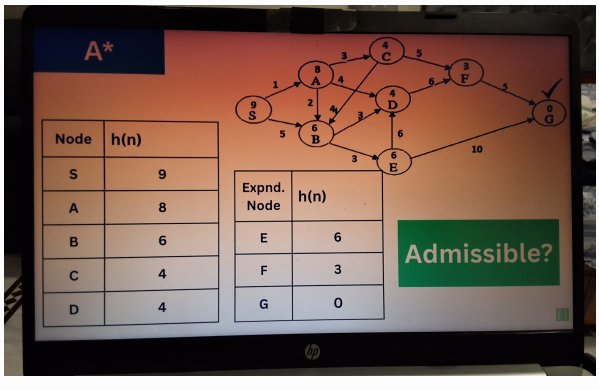
only optimal if h(n) → admissible

h(n) is only admissible if it never overestimate actual cost from start to goal

Ids.



guarantees: terminated. found goal node of minimal path



check for each node

H(B) = 6

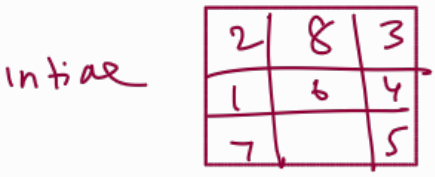
Admissible

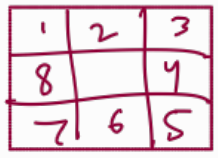
Actual cost = B → E → G
= 13

or

B → D → F → G
= 24

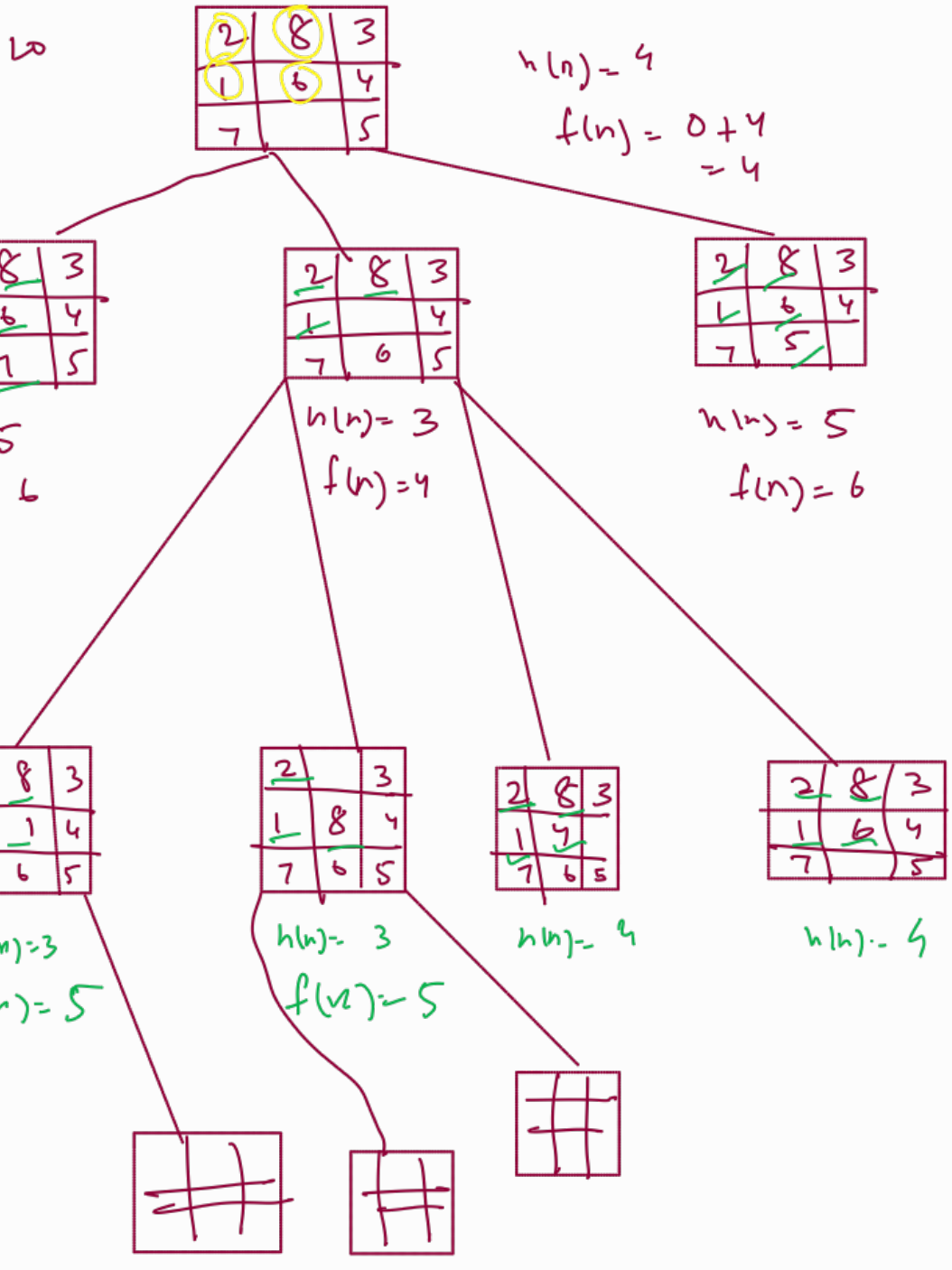
(A* Search algo) on 8 puzzle



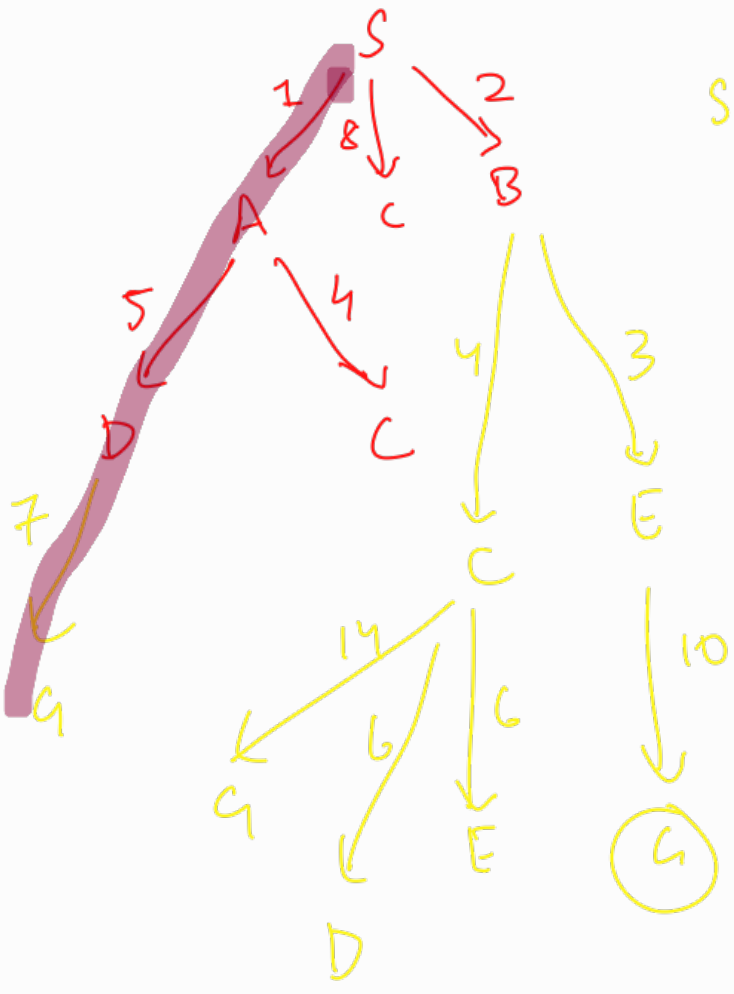


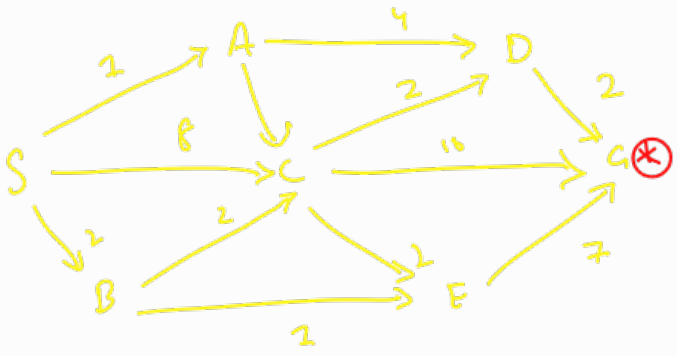
goal

given
g(n):- depth of node
and h(n) - no of misplaced tiles.



ucs Assg 1)





(A,1) (G,8) (B,2)

(D,5) (G,4)

(G,4) (E,5)

(G,10)

(D,6) (E,6) (G,14)

(G,7)

keep going until we get min cost goal

---

explore this

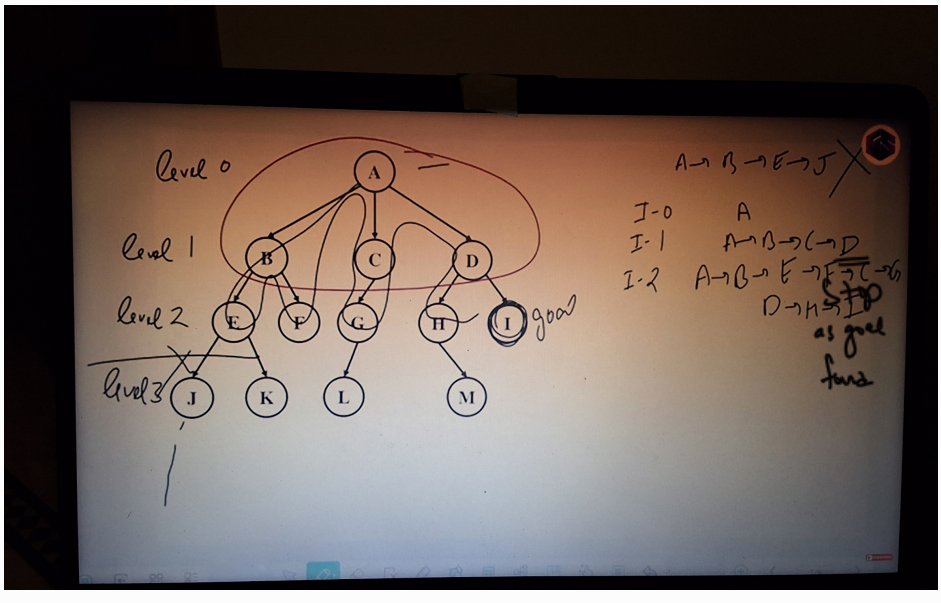
(A,1) (B,3) (C,4)

(D,5) (E,6)

☆ BFS → takes more storage and is complete

☆ DFS → lesser storage But can go to infinite branch
Searching here not complete

IDS → Searches limit by limit
ensures both completeness and optimality
↳ ensure exploration of all nodes before going deeper.
(L→R)



Problem with IDS is that it repeats nodes

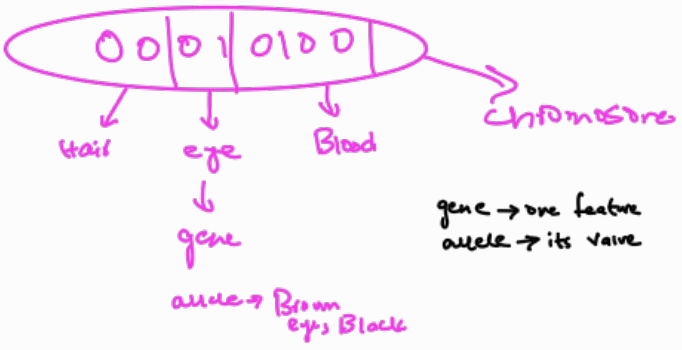
(GA) → Many Students
→ Test them with a problem
→ Best students pass
→ combine them and create better students.

Chromosome → way to represent a sol e.g 10110101
population size → no of sol

tools to create new sol → crossover, mutation

generation gap → How much of the poplution is replaced each time

elitism → keep the Best Solutions safe



candidate sol → 1 possible answer
population size → total no. of candidate sol

population → collection of chromosomes.

Chromosome = x y z → gene

x = 101
y = 111
z = 011

allele is value

101 → 5

Population size → Too large (slow progress) Takes time to process.
" → Too Small (Low diversity) → may get Bad Sol.

More diversity → Better chance of finding a better Solution.

Pop... size Should be balanced

Fitness function → way to determine how good a Sol is.

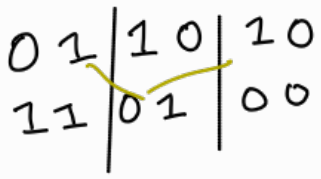
GA uses fitness function to choose good Sol and remove the bad ones.

Reproduction operators:: (create new Sol from old ones)

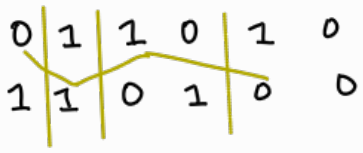
1) cross-over
↓
combines good feature from parents.

01 00 000 C1 → length must remain same as of parents
11 100101 C2

2) Mutation (Small change)
↓
Adds variation
Prevents Sol from becoming Similar



2point crossover



0 1 0 1 2 0 c1
1 1 1 0 0 0 c2

0 1 1 0 0 0 c1
1 1 0 1 1 0 c2

creates more diversity

Mutation Rate → chance of mutation (Random value)

Low ρ → new traits appear slowly | Too rigid

High ρ → each generation will be unrelated to Previous gen. | introduces Random.

new traits → new possibilities.

(Selection Algorithm) → which sol (chromosome) should be chosen as parents.

Selective pressure → Rate at which Best Solutions are selected.

Low SP → Bad sol also get selected | both bad and good sol treated equally.

High SP → only Best sol selected.

Types of Selection Algo

→ Random (no use of fitness score) weak

→ Proportionate (Based on fitness) (Roulette wheel) Biased

→ Preserves genetic diversity
→ Dynamic S.P, Drawbacks → Stagnation at convergence

⇒ Linear Rank (Rank all sol from Best → worst) stable

↳ assign Prob.

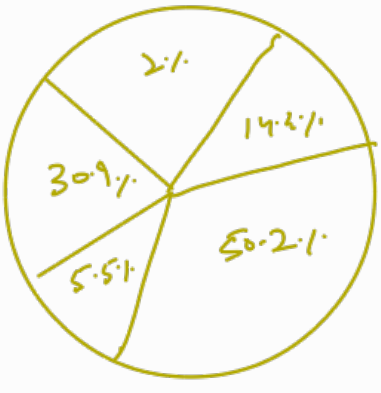
⇒ Tournment (popular)

compare Best within few sol

$$P = \frac{\text{fitness}}{\text{total fitness}}$$

note:: Selection of only the most fittest Chromosones may result in loss of rare gene which may be present in less fit member.

to overcome this.



High diff b/w Smaller and larger fitness means High S.P.

if population is Similar fitness values then low S.P. hence hinder in finding better sol.

Linear Rank

Prevents loss of Useful genes.

Solution

note:: if fitness = 0, Probability = 0
hence it will never be selected.

But it can contain useful gene

this was problem in Roulette wheel

chromosm	fitness	Rank	new fitness values
A	50	1	—
B	40	2	—
C	10	3	—
D	0	4	—

↓ use this

1) Sorted from Best fit → worst
2) each chromsome is assigned a new fitness based on

$$(P-r)+1$$

P → most chromosomes
r → rank

$$\text{new fitness} = (P-r)\frac{(max-min)}{(P-1)} + min$$

user adjusted.

↳ used when want to control Selective pressure

Strong Sel → Larger gap b/w max and min.

Weak Sel → Smaller gap b/w max and min.

	Probability of getting selected.	Rank
A	0.6	1
B	A fails = 0.4 × B success = 0.6	2
C	0.4 × 0.4 × 0.6	3
D	0.4 × 0.4 × 0.4 × 0.6	4

$(1-P) \times (P)$

↓ failure ↓ success

Linear Rank

Adv :: constant S.P

Dis:: unfair to ties ↓

if 2 chromosomes have exact same fitness score one is ranked higher, meaning they will recieve diff Ep

Roulette wheel:: proportionate algo.

Chromsone	(f)	Probability	cumulative propability	class boundaries
C1	7	7/14 = 0.50	0.50	0 – 0.50
C2	4	4/14 = 0.28	0.78	0.51 – 0.78
C3	2			
C4	1			
	14			

Q Random no give → P = 0.35 → falls in 0.0 → 0.50

↳ C1 wins

Tournment Selection:

Instead of checking all chromosome, pick a small group (size=b) and select best from there.

A, B, C, D, E S = 2

B and D → Choose the fitter one.

Small Size	Large Size
weak and strong both can win	Strong wins

Both chromosomes

Problem: if there is a highly diverse chromosome, but that has a slightly weaker score, it will get wiped out by better score chromsome. Hence cause Diversity problems.

sol

Diversity : How diff a chromosome is from others.

A : 90
B : 85
C : 80

population becomes to similar

Already Selected

11111111

Measuring diversity $\Sigma(Dis)^2$

Large value → very diff
Small → low.

Candidate

A 11111100 fitness = 6

high fitness low diversity

B 00000011 fitness = 6

High diversity

Here our unique candidate now cannot be replaced by High fitness leader.

1)

A : Diversity : Differs by 2 bits

$(2)^2 = 4$

B : Diversity = $(6)^2 = 36$

2) Assign Ranks → Based on fitness

Rank 2 A

Rank 2 B

3) Diversity Rank

1) B
2) A

Total Rank:
for
A = 1 + 2 = 3
B = 2 + 1 = 3

To find optimal fitness: find derivative and Plug 4 0.

Equation → $x^3 - 60x^2 + 900x + 100 = 0$

$3x^2 - 120x + 900 = 0$

$x = 30$ $x = 10$ Plug Back in

at $x = 10$ → 4100 LTop

at $x = 30$ → 100 (Bottom)

elism = 0.1 P = 20

locked in = $20 \times 0.1 = 2$ individuals

Q1 P = 50, 5 fittest are placed directly for next gen. The other 45 spots are chosen from a pool. 30 offspring mutate it to new gen.

elism: $\frac{5}{50} = 0.1$

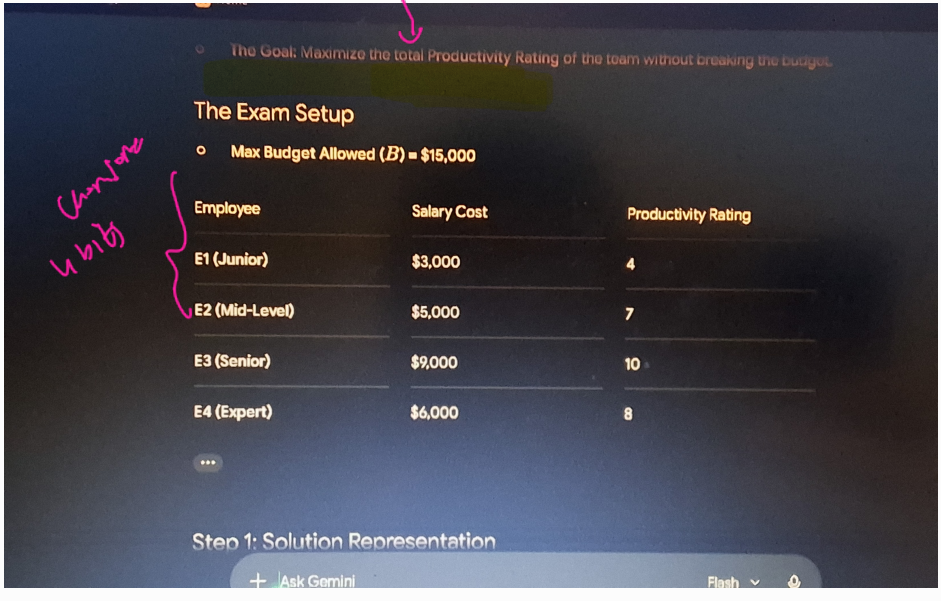
g.f = $\frac{\text{new members replaced with}}{\text{total pop}}$

$\frac{30}{50} = 0.6$

note: when g.A runs smoothly it can duplicate chromosomes with highest scores, so the Algo must terminate duplicates

---

Knapsack — cannot exceed a certain limit else chromosome gets discarded.



Chromosome Length

1: employee hired

0: not hired

C1 : 1101

Total cost = 14000$ ✓

Total fitness = 19 ✓

C2: 0111

cost = 20000 ✗

fitness = 0

next apply selection algo

---

when threshold replaces Bias

$\theta = 0.5$

$y_{out} \geq 0.5 \rightarrow 1$

$y_{out} < 0.5 \rightarrow 0$

Supervised learning: using datasets to train (learn from e.gs.

- Regression: continuous output
- Classification // Discrete

Regression → Relation b/w 2 things.

Simple Reg 1 Input → output

Multi Reg Multi-Input → output

$y = b_1 x + b_0 + \epsilon$ → Error. (Actual − predicted) $\epsilon = y - \hat{y}$

- y: Actual val
- $b_1 x$: input; slope
- b_0: y-int; Starting value when $x=0$

No relationship $b_1 = 0$

$\hat{y} = b_1 x + b_0$

gives predicted val

Line of Best fit goes in middle and is the Best prediction line

$error = d_1^2 + d_2^2 + d_n^2$

(Actual point − line)

Smallest error → line of Best fit.

Loss function Mean Square Error

$$= \frac{\Sigma (y - \hat{y})^2}{n}$$

error = $y - \hat{y}$

tells how far our predictions are from real values on Avg

↓ MSE = Better model.

D → Squares

---

finding line of Best fit / Least Square

$$\hat{y} = b_1 x + b_0$$

$\bar{x}$ → Avg

$\bar{y}$ → Avg

$$b_1 = \frac{\Sigma (x - \bar{x})(y - \bar{y})}{\Sigma (x - \bar{x})^2}$$

↓

Slope

$$b_0 = \bar{y} - b_1 \bar{x}$$

Variability:: ⟶ How Spread out our data is.

↓ ↗ ↘

Range Variance S.D

L.V → consistent, predictable

H.V → Random, scattered.

coefficient of determination::

↓

measure how good our model is

$$\Sigma (y - \bar{y})^2 = \Sigma (\hat{y} - \bar{y})^2 + \Sigma (y - \hat{y})^2$$

SSR → Sum of Squares due to regression

↓ total Spread SST

↓ explained Spread (the part model accurately predicts)

↓ Error SSE → Sum of Squares due to error

$\frac{SSR}{SST}$ or $\frac{1-SSE}{SST}$

$R^2 = \frac{\text{explained part}}{\text{total}} = \frac{\Sigma(\hat{y}-\bar{y})^2}{\Sigma(y-\bar{y})^2}$

How good our model predicts data / variation in our data.

$R^2 = 0$ (useless)

$R^2 = 1$ perfect

$R^2 = 0.75$ (explains 75% of data accurately)

$R^2 = 0.8772 \times 100$

$\rightarrow$ 87.72% is predicted accurately the remaining is SSE / noise

For a good model

$\rightarrow$ x and y should follow a trend. greater will be R^2

$\rightarrow$ if x and y don't follow a trend, points Scattered, Low R^2.

Sample correlation coefficient | Relationship b/w x and y

$r_{xy} = \text{Sign of Slope } (b_1) \times \sqrt{R^2}$

↓ coefficient of determination

means linear relationship only

$r = 1 \rightarrow$ x and y Inc tog

$r = -1 \rightarrow$ 1 inc other dec

$0 < r < 1$ pos but not perfect.

$r_{xy} = +\sqrt{0.8772} = +0.9366$ → Pos correlation b/w x and y but not perfect

$$m = \frac{\Sigma (x - \bar{x})(y - \bar{y})}{\Sigma (x - \bar{x})^2}$$

↓ slope

$$y\text{-int} = \bar{y} - m\bar{x}$$

Square the calculated vals then add.

$$\frac{\Sigma (\hat{y} - \bar{y})^2}{\Sigma (y - \bar{y})^2}$$

$$r_{xy} = \text{Sign of slope b/l} \times \sqrt{R^2}$$

Artificial neutral networks (ANN) → human brain

↓

work on brains parallelism.

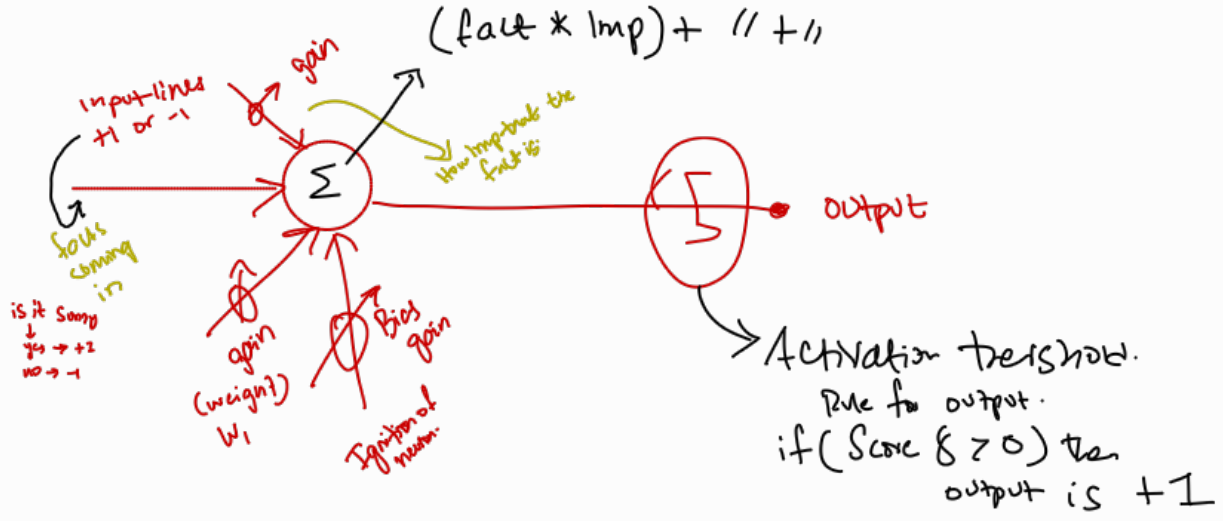
Sequential

↓

computer checks step by step

Parallel

↳ brain looks at once, but due to massive parallelism it does task fast, even though neurons are very slow.



AND

-2 → 0 | false
1 → 2 | True

x_1	x_2	y
-1	-1	-1
-1	1	-1
1	-1	-1
1	1	1

Bipolar

no of Rows = no of neurons

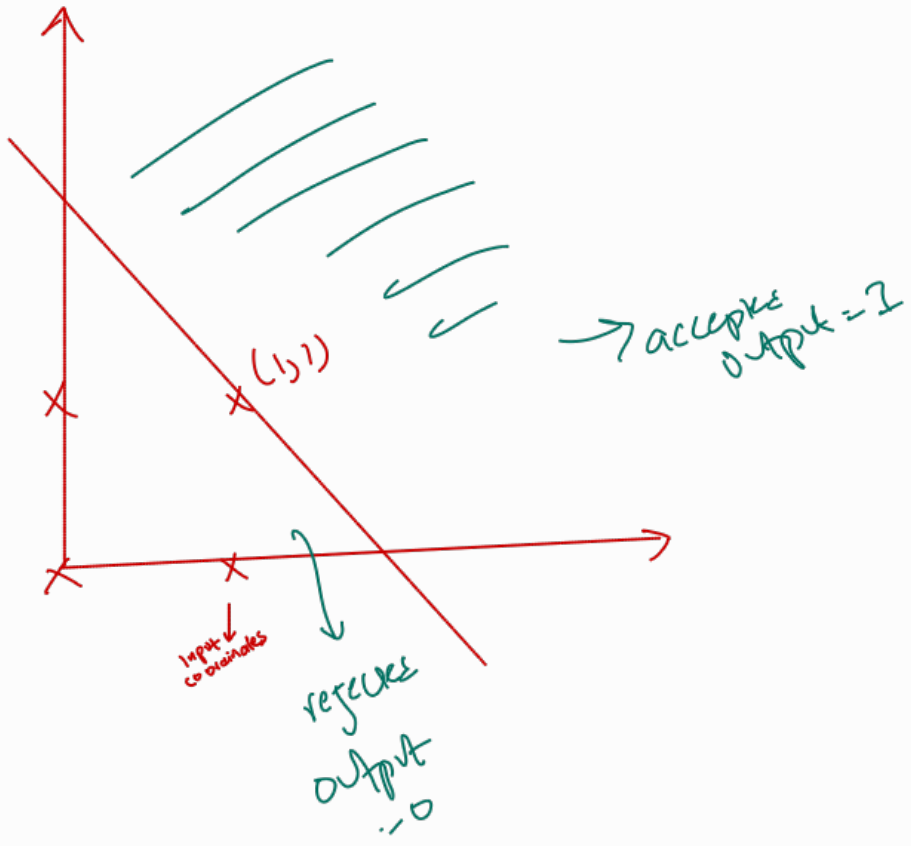
Perceptron d-y → values must be same

let

x_1	x_2	w_1	w_2	Σ	y
0	0	1	1	$x_1w_1 + x_2w_2$ 0	0
0	1	1	1	1	0
1	0	1	1	1	0
1	1	1	1	2	1

threshold 0.2

(1,1) → Pass

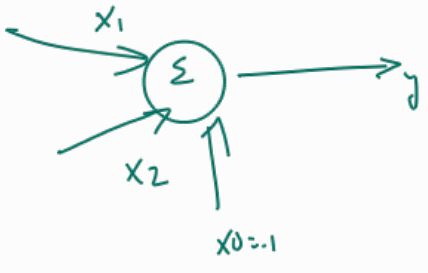


(Typical ANN) $X_0 \rightarrow$ Bias. (Set to -1)

Instead of having a rule (threshold 0) we put a extra input in neuron called Bias.

if total sum > 0 → new goal

→ then activate



$y = b + w_1 x_1 + w_2 x_2$

↓

Bias

typical ANN we just add a Bias instead of 0

where $y > 0$ to activate

Hebb Rule → Neurons that fire together wire together.

Perceptron → A better way to learn until perfected the results.
if there is still a diff then neuron has not learned.

Activition functions → Σ

↓

tells how to turn the total score into final output

Binary | Bipolar → A yes & no.

Sigmoid → Smooth curve.

Hyperbolic → curve −1 → +1

$$y = x_1w_1 + x_2w_2 + b$$

Perceptron Learning:

based on activation fun

x_1	x_2	w_1	w_2	d	y	y_a	Δb	Δw_1	Δw_2

previous weights

if d and y_a is not same update the previous weights.

$\Delta w_1 = \alpha d x_1$ (desired input) α → learning rate

$\Delta w_2 = \alpha d x_2$

$\Delta b = \alpha d$

↳ add these Δweights to previous weights to get

error: $d - \hat{y}$

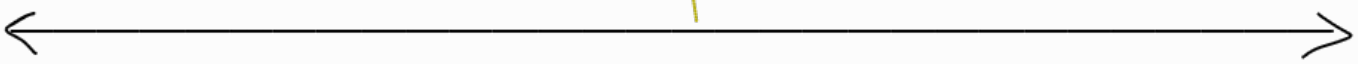
$\hat{y} = x_1 w_1 + x_2 w_2 + b$

$\Delta w = d \cdot Error \cdot x_i$

$w_{new} = w_{old} + \Delta w$

$b_{new} = (\eta \cdot error) + b_{old}$

new weights



Add Structure

input layer (takes data) → Hidden layer (process data) → output layer give result

Types of neural networks

1) Feed-forward

Data moves in 1 direction only.

1) Single layer
2) Multi layer

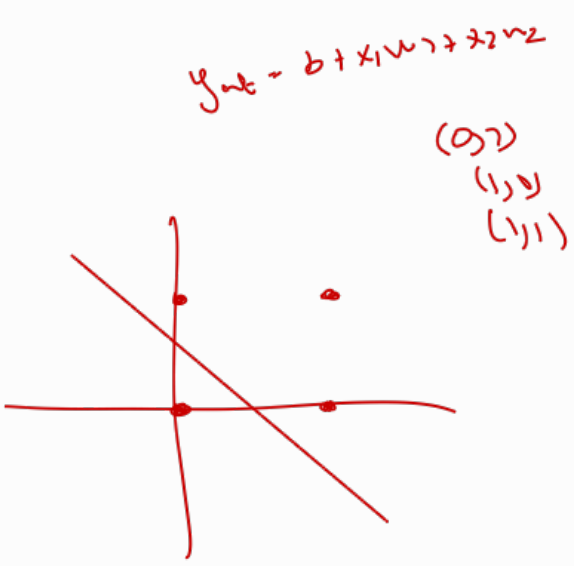
Types of training algo → that helps Learning of network. Not make mistakes

→ Supervised (Answer already given)

↳ Learn with answers | output already given it just compares with input

→ Unsupervised

↳ Learn via patterns



Single neuron → 1 line only drawn

$$y_{act} = w_1x_1 + w_2x_2 + b$$

$$w_1x_1 + w_2x_2 + (w_0 \cdot x_0)$$

↓

Bias input always = 1.

Perceptron Learning:. Learning Rate : 0.1 → α

x_1	x_2	w_1	w_2	b	y_{act}	$\hat{y}_{act}$	d
		1	1	1			
1	1	1	1	1	3	+1	1
1	-1	0.9	1.1	0.9	1	+1	-1
-1	-1	0.9	1.1	0.9	-1.1	-1	-1
-1	1	0.9	1.1	0.8	1.1	1	1

$$y_{act} = b + x_1w_1 + x_2w_2$$

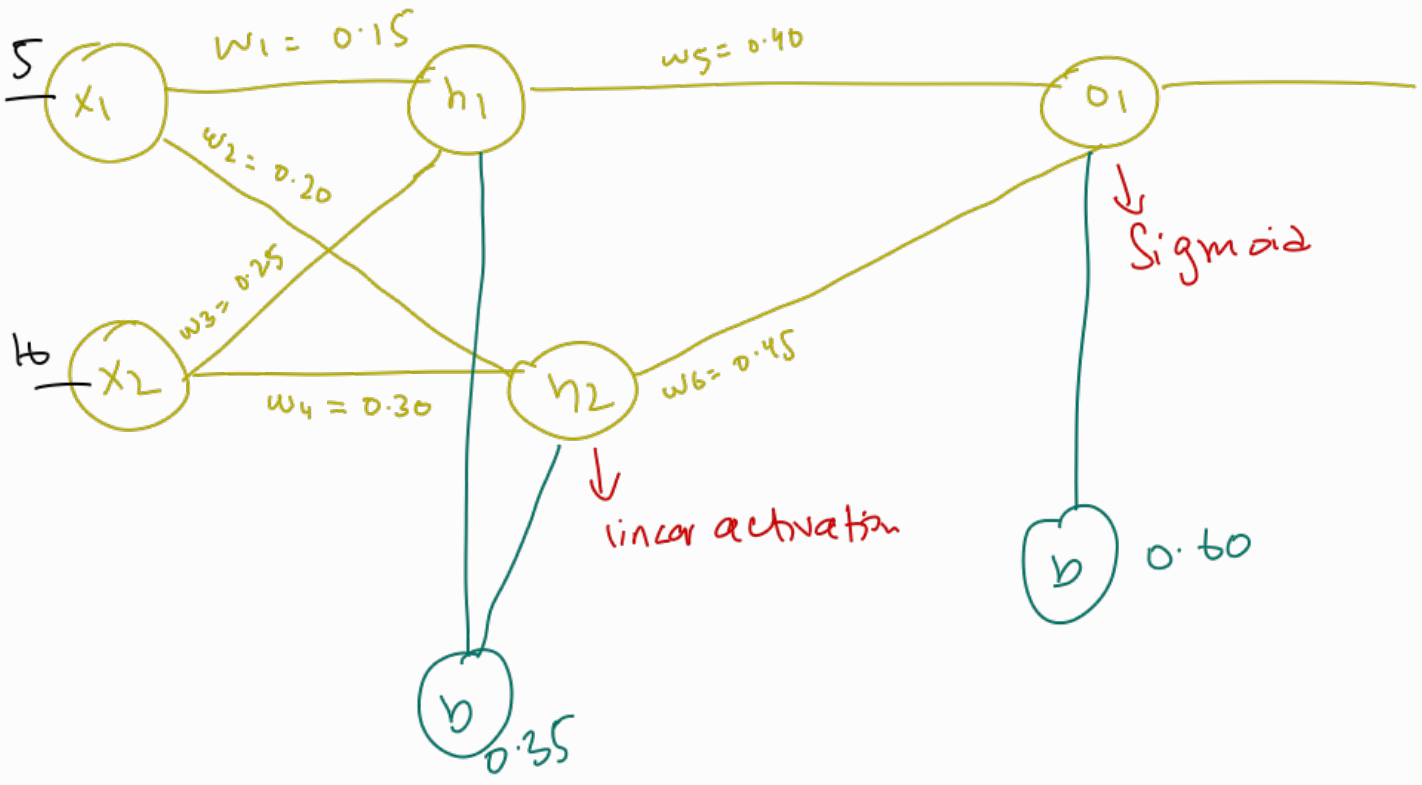
↓

Sum

$$\Delta w_1 = \alpha(d - \text{actual}) x_1 + \text{Previous weight}$$

$$\Delta b = \alpha d$$

Feed forward activation:



calculating outputs for h_1

$$y_{act} = x_1 w_1 + x_2 w_2 + b$$
$$= 5 \times 0.15 + 10 \times 0.25 + 0.35 = 3.6$$

$$\text{output} = \frac{1}{1 + e^{-(y_{act})}}$$

$$h_2 = 0.20 \times 5 + 0.30 \times 10 + 0.30 = 4.3$$

calculating o_1:- $3.6 \times 0.40 + 0.45 \times 4.3 + 0.60$

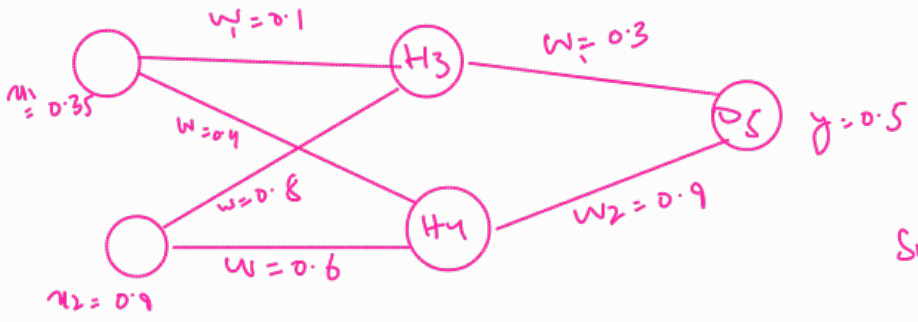
$$= 3.975$$

$$\frac{1}{1 + e^{-1}(3.975)} = 0.981$$

Backpropagation:

→ create feed forward network and solve normally.
→ now Backpropagate to remove errors and update

$\alpha = 1$ Target output $= 0.5$



Sigmoid $= \frac{1}{1+e^{-n}}$

$H_3 = (0.1 \times 0.35 + 0.8 \times 0.9)$
$= 0.755$

$H_4 = (0.6 \times 0.9) + (0.4 \times 0.35)$
$= 0.68$

now at O_5

$(0.755 \times 0.3) +$
(0.9×0.68)
$= 0.8385$

$\hat{y} = \frac{1}{1+e^{-}(0.8385)}$
$= 0.698$ (network output)

error = $y_{target} - y_{actual}$
$0.5 - 0.698 = -0.198$

now update the weights:

new weight = old weight $-$ (α * weight gradient)

new Bias = old Bias $-$ (α * Bias gradient)

δ = error × actual output × (1 − actualoutput)

calculate this at output

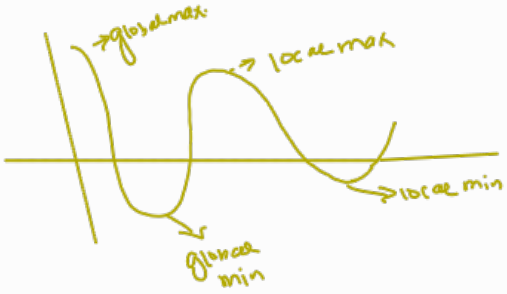
$\Delta w = \alpha \cdot \delta \cdot$ output at that neuron + old weight

local search → optimization search used when the path itself is not important but the goal. Problem: Local optimum

Hill climbing: → Starts at Random and only moves to next neighbour if sol is better.

if more neighbours are better, algo stops and declares current sol as goal

no guarantee it will reach the sol.

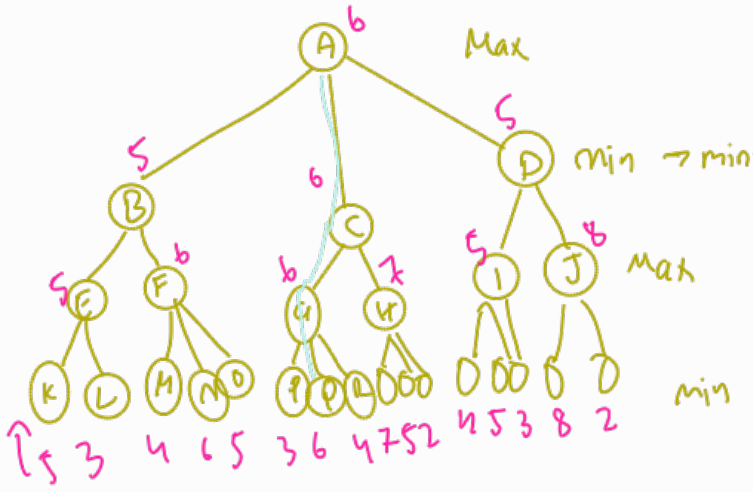


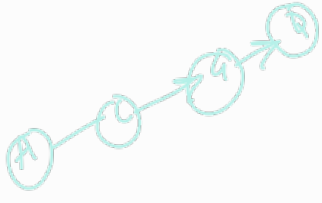
Initially use a Random State

Optimum means either max or min
So to reuse that in hill climbing, Run multiple times with diff random points and pick best sol

---

Adversarial Search → used when agents are moving moves and countering your moves.





Max lose → -1
min lose → 1

→ FC layer

Total: hidden x (input +1) + (output x (hidden +1))

(F x F x In) +1 * K

(3 x 3 x 1) +1 * 10

= 100

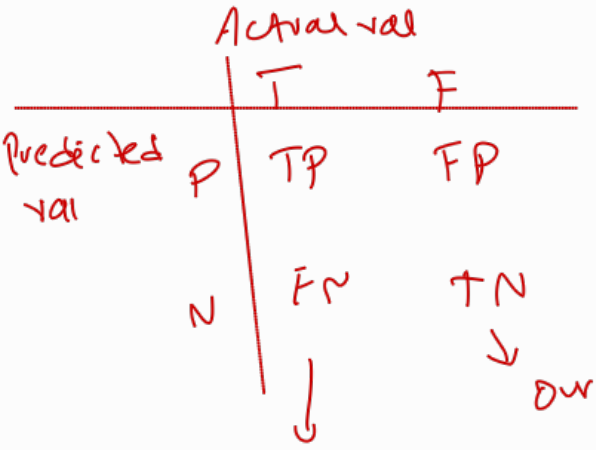
K: 10
F: 3

- Features of a nn are the no of inputs.
- no of weights in a nn are the no of connections

Performance measures: → Binary class

A.V → T | F

P.V → P | N



TN → our model correctly predicted the value that is same as actual value

FN → our model predicted a neg value even though it was false

Accuracy: $\frac{\text{correct predictions}}{\text{Total}} \rightarrow \frac{TP+TN}{TP+TN+FP+FN}$

only works if dataset is not unbalanced → Shows overall model performance

if we have a rare disease (only 5%) and model says everyone is healthy → this is useless

Recall: $\frac{TP}{TP+FN}$

0% → means our model caught 0 actual cases. (Underperforming)

100% → our model caught every single case

Use accuracy when data is balanced 50|50

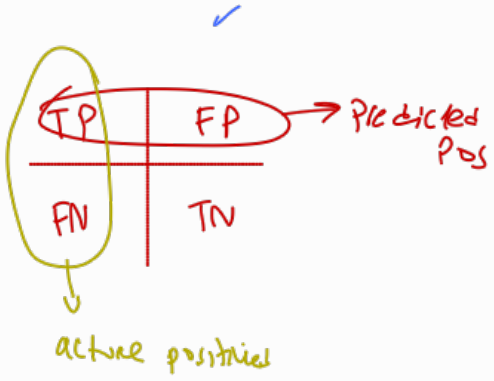
Precision: $\frac{TP}{TP+FP}$ → how many predicted positive cases are actually positive

Balanced dataset → where our model is trained on equal distribution of classes.

20% no tumor
20% T1-type
⋮

→ this is great because our model doesn't develop a bias for 1 class. Hence effecting Accuracy

Models real performance is measured on data that it has never seen / test dataset.



Predicted > actual → low Precision
actual > Predicted → missing many real cases, Low Recall.

F1-Score: $2 * \frac{Precision * Recall}{Precision + Recall}$

1 (Best Score) OR 0 (Totally failing)

→ Use when we have a unbalanced d.s.

High recall ↑ low precision ↓

High precision ↑ low Recall ↓

Multi-class confusion matrix:

more than 2 categories to predict

Predicted \ Actual			
	TP		
		TP	
			TP

TP → diagonal → Total correct predictions

Accuracy: $\frac{TP}{Total}$

---

calculating: FN for a class

	S	B	C
S		2	3
B	9		7
C	11	12	

for S class

(FN) 2+3 = 5

FP for a class is sum of corresponding columns.

	S	B	C
S		2	3
B	9		7
C	11	12	

S → 9+11 = 20

calculating TN

	S	B	C
S	5	7	4
B	0	1	17
C	0	0	11

for S :

17 17 + 0 7 4

= 29

Macro Averaging: Used for multi-class only

↳ all classes assigned equally

1) compute precision, Recall, F1 Score for each class
2) Add them together
3) Divide by no of classes

→ $\frac{\text{Precision}_{\text{class A}} + \text{Precision}_{\text{class B}} \ldots}{n}$

→ $\frac{\text{Accuracy}_{\text{class A}} + \text{Accuracy}_{\text{class B}}}{n}$

use: When u care abt how your model performs on every class equally in a unbalanced dataset.

Micro Averaging: → used when needs to calculate overall performance

↓

there the class with better score will dominate

dataset is balanced.

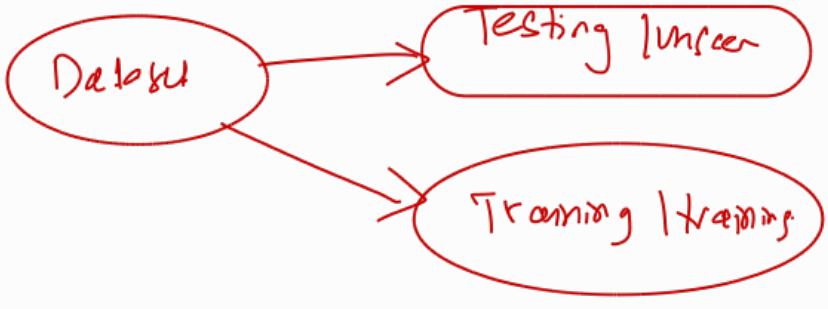
No categories like in Macro Averaging - (worst overall accuracy as it counts as whole)

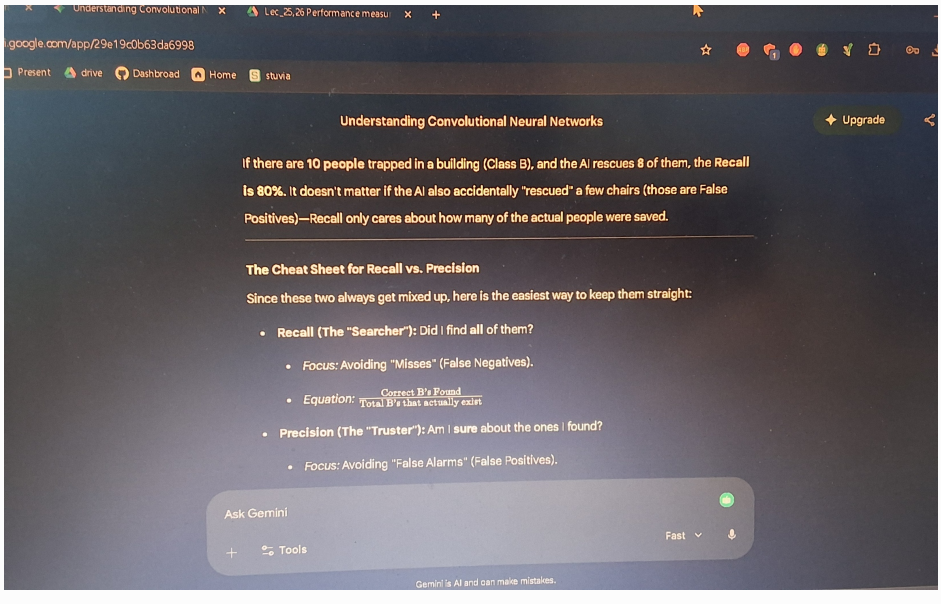
Micro Precision

$\frac{\sum TP}{\sum TP + \sum FP}$

↓

for every class





CNN — computer understands a image via edge detection.

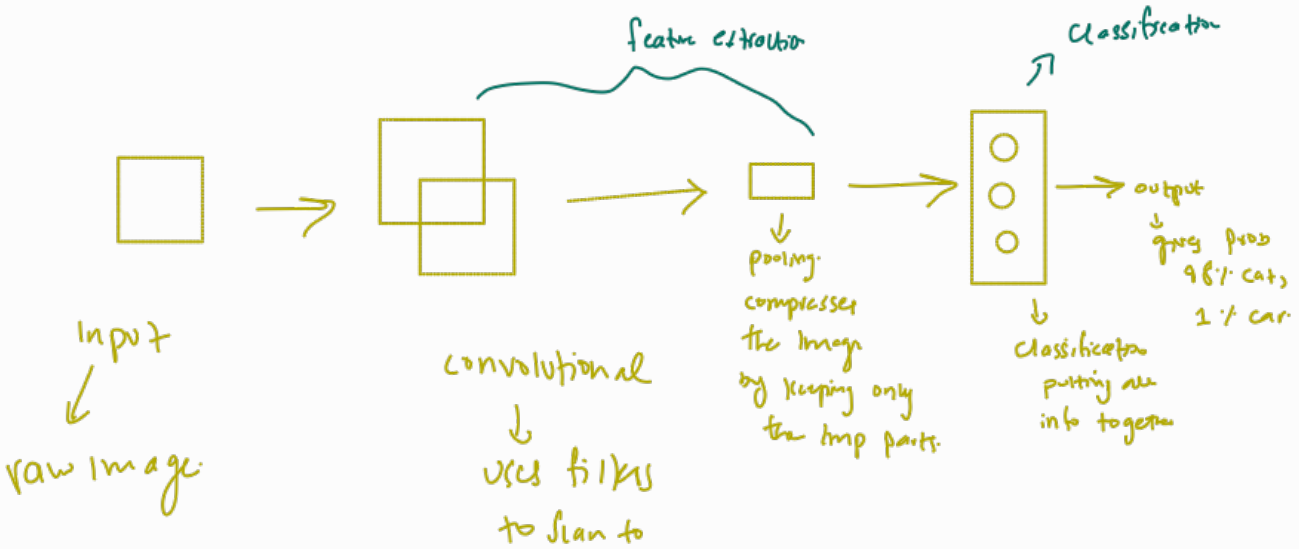
vertical edges + horizontal edges in a image

↓

helps in

recognising objects in images / non-image data e.g. audio

finds edges → finds shape → objects



input → raw image

convolutional → uses filters to scan to screen for edges

+ RELU → takes all negno resulting from convolution and turns to zero. keeping positive singles only.

pooling → compresses the image by keeping only the imp parts.

classification → putting all into togetn

output → gives prob 98% cat, 1% car

3 main layers

1) convolutional uses a filter

2) Pooling

Shrinks image → helps AI avoid unnecessary detail

3) FC (fully connected)

↓

classification

output size of conv layer

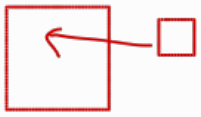
$$\frac{W - F + 2P}{S} + 1$$

W → width / height of the image

F → filter size

P → Padding

S → Stride

Kernel / filter 

Stride → how many steps the filter takes as it moves over the image

Padding → preserves the original image dimension after applying kernel / to edge loss

Small stride → high detail but takes longer time

Large stride → gives the general idea, faster, less mem...

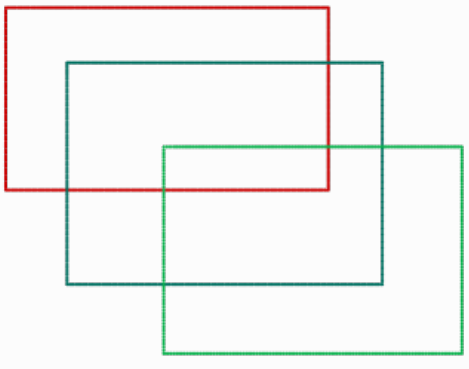
Size after each layer ↓

$$\left[\frac{n+2p-f}{s}+1\right] \times \left[\frac{n-2p-f}{s}+1\right]$$

→ Same formula.

Multiple filters: → used to check multiple features of the images at once, vertical / horizontal edge etc. / parallel processing

Multichannel Image →



Actual size ~ 3×3×3 → channel

So filter must have same no of channels as input

3 channels

Pooling layer:: → makes image smaller while keeping the most imp info.

- Reduce size
- Prevents overfitting / discard unnecessary details.

Types

Max pooling:: → highlights only the prominent

Avg pooling → Smooth out the details

Both use 2x2 window

$$\frac{W - F}{S} + 1$$

(n)

there is no padding (P=0) in pooling and we don't need to preserve the image

FC → fully connected Layer → classification → makes final decision.

Before classification → flatten the data. 2D → 1D

Because classification layer is made of neurons that don't understand 2-D.

Convolutional layer is much more effient than neural network. also. (fully connected)

→ Deals with images in 2-D

→ Fewer weights to store

→ Faster

→ uses one learning rule for whole image

total error goal → must be close to 0.

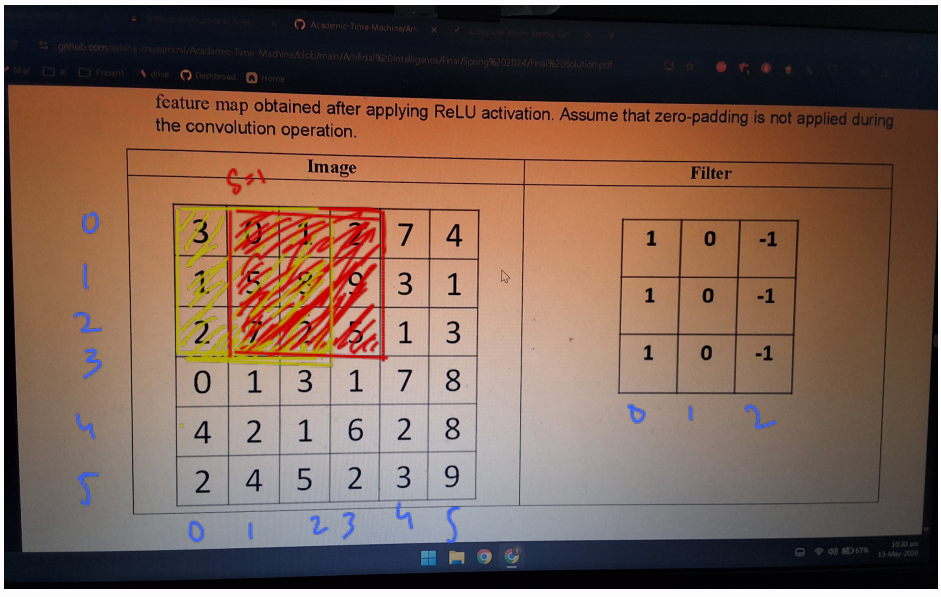
$$J = \frac{1}{m} \sum_{i=1}^{m} L(\hat{y}, y) \rightarrow L|\hat{y} - y|$$

m → total no of images

$\hat{y}$ → pred

y → actual

how wrong our neural network is across entire training session.



feature map obtained after applying ReLU activation. Assume that zero-padding is not applied during the convolution operation.

Image

3	3	1	2	7	4
1	5	2	9	3	1
2	3	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

Filter

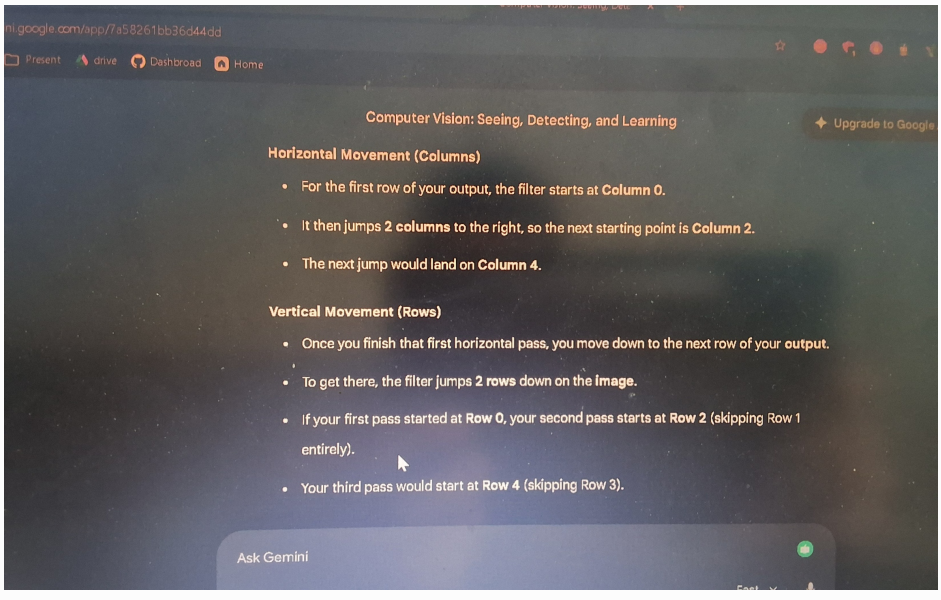
1	0	-1
1	0	-1
1	0	-1

S=1 move 1 column at a time.
After the first row is done move to second row of the image and apply filter.

if S=2 → move 2 cols
→ move 2 rows

flattening:

$$\begin{bmatrix} 0 & 9 \\ 1 & 0 \end{bmatrix} \rightarrow \begin{bmatrix} 0 \\ 9 \\ 1 \\ 0 \end{bmatrix}$$



Computer Vision: Seeing, Detecting, and Learning

Horizontal Movement (Columns)

- For the first row of your output, the filter starts at **Column 0.**
- It then jumps **2 columns** to the right, so the next starting point is **Column 2.**
- The next jump would land on **Column 4.**

Vertical Movement (Rows)

- Once you finish that first horizontal pass, you move down to the next row of your **output.**
- To get there, the filter jumps **2 rows** down on the **image.**
- If your first pass started at **Row 0,** your second pass starts at **Row 2** (skipping Row 1 entirely).
- Your third pass would start at **Row 4** (skipping Row 3).

Ask Gemini

Important points: pooling is used to shrink/compression the image, but if we use Stride >1 it automatically shrinks the image anyway so we can skip the pooling layer.

Why CNN's are more efficient than Regular NNs

Local receptive fields → cnn neuron focuses on a small patch

organizes neurons by using feature maps.

Feature Map 1	Feature 2
↓ looks for horizontal edge	looks for vertical

Clustering

↓

classification of diff groups / partitioning of dataset into subsets based on common trait /

Act of grouping similar things together

Supervised Learning vs unsupervised

↓

AI model already knows the answers alongside the data. It just trains on that data. / labels included.

↓

finds patterns on its own. via common ality

Types of clustering.

1) Hierarchical

→ Agglomerative (Bottom up) clustering

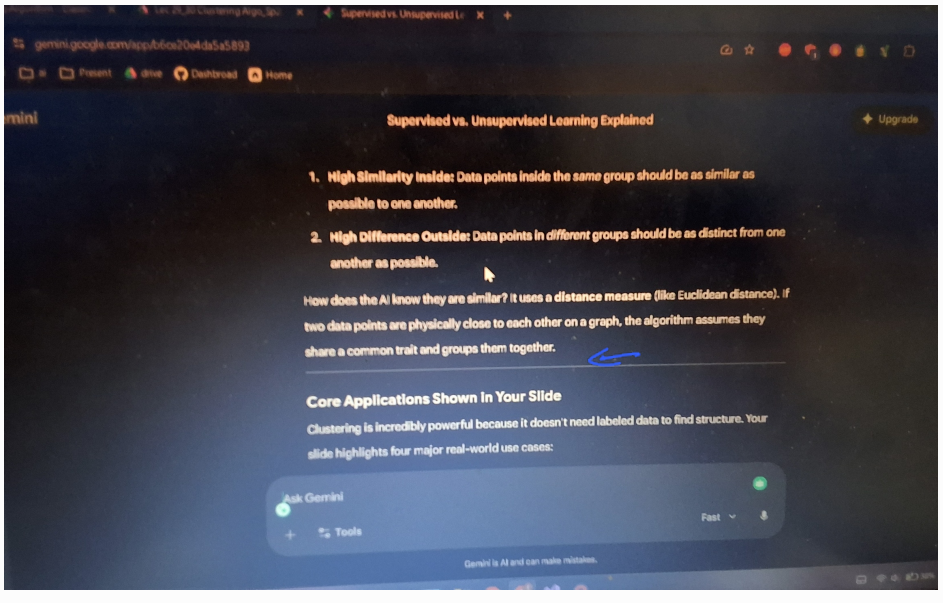
In this algo looks for the 2 closet points and merges them. Next it merges the next closests points until everything is 1 cluster

Divisive (Top-down)

↓

Starts with all the data points trapped inside a single cluster. Looks for disagreements and splits data until individual.

Major flaw :. we cannot correct mistakes, once the algo makes a bad merge / split it permanently locked into that path



2) Partitional

- k-means
- fuzzy c-means

You tell the algo, how many groups you want (k).

It places centers in dataspace and every point joins whatis closest to that

→ Struggles in large datasets especially if its not clean.

3) Density Based

→ looks for crowded data / data tightly packed and becomes part of cluster, Ignores outliers.

euclidean dis

$$d(x,y) = \sqrt{\sum (x-y)^2}$$

Manhattan dis

$$d(x,y) = \sum |x-y|$$

$$= |x_2 - x_1| + |y_2 - y_1|$$

(1,2) (4,6)

$$d(x,y) = \sqrt{(x_2-x_1)^2 + (y_2-y_1)^2} \ldots$$

$$= \sqrt{(4-1)^2 + (6-2)^2}$$

Maximum Norm (1,2) (5,5)

$$d(x,y) = \max(|x_2 - x_1|, |y_2 - y_1|)$$

$$= (|5-1|, |5-2|)$$

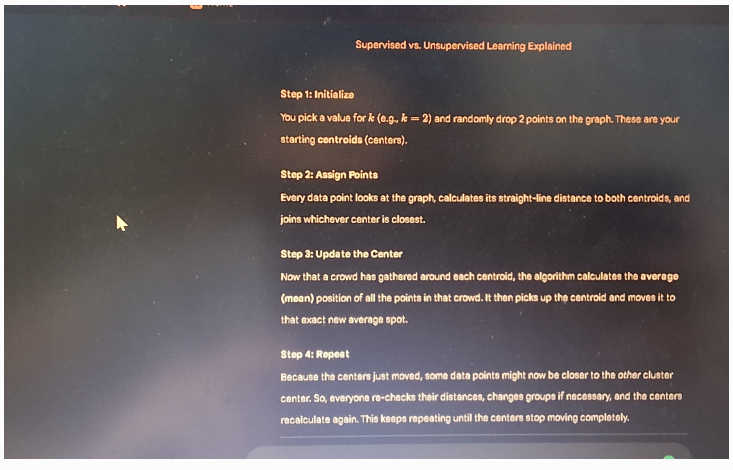
$$= (4,3) \rightarrow 4$$ → total time to reach a destination is determined by largest no only

K-means

k<n → noof groups < total noof datapoints

→ uses euclidean distance

(noof groups = noof centroids)



K-means

1) Randomly plot centroids
2) Assign datapoints to those centroids
3) find new centroids by calculating Avg of all points excluding old centroid.
4) keep on doing until centroids loc become fixed.

→ No overlapping groups / mutually exclusive → every single data set belongs to 1 group only

→ Spherical shapes / neatly aligned → dream dataset for k-means

Rather than initializing Random centroids initially.

Pick actual points from dataset → To Reduce Randomness

However can run into local optima problem.

meaning 2 initial centeroids take and split 2 group for themselves leaving other groups behind

So) Run → multiple times. | pick centroids in your dataset that are far apart from each other to avoid greedy

(Implementation)

Individual	variable 1	variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
⋮	(x)	(y)

→ picking centroids from our own data set.

now calculate distance

C1(1,1) C2(5,7)

for individual 2 : (1.5,2)

$$\sqrt{(1.5-1)^2+(2-1)^2} = 1.12$$

✓ ind#2 goes to C1.

$$\sqrt{(1.5-5)^2+(2-7)^2} = 6.10$$

(now update these mean position) | new centroid.

↓

Looking at image.

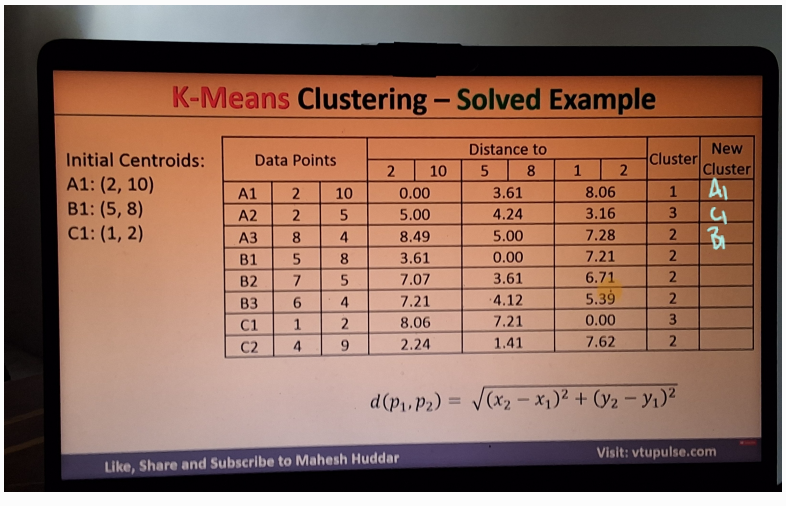
new centroids

A1(2,10)

B1 → $\left(\frac{8+5+7+6+4}{5}\right)$ → x

$\left(\frac{4+8+5+4+9}{5}\right)$ y

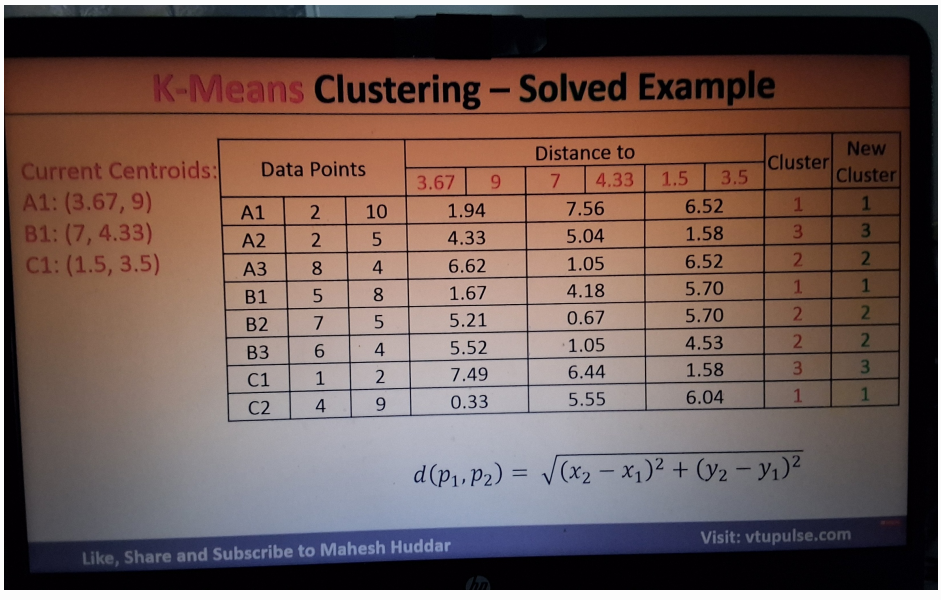
C2 → $\frac{(2+1)}{2}$ → x $\left(\frac{5+2}{2}\right)$ → y



new centroids

A1 (2,10)
B1 (6,6) → Repeat Step 2
C1 (1.5, 3.5)

Keep on doing until the groups converges



Evaluation function of k-means :- Low SSE → great k-means, groups are tightly packed.

High SSE → Bad, loose points

if you want to lower you error in k-means just add more centroids. ↑↑K But can cause over fitting

calculating SSE :-

Take every point distance from its own centroid and Square it..

K-Means Clustering – Solved Example

Current Centroids:
A1: (3.67, 9)
B1: (7, 4.33)
C1: (1.5, 3.5)

Data Points			Distance to 3.67, 9	Distance to 7, 4.33	Distance to 1.5, 3.5	Cluster	New Cluster
A1	2	10	1.94	7.56	6.52	1	1
A2	2	5	4.33	5.04	1.58	3	3
A3	8	4	6.62	1.05	6.52	2	2
B1	5	8	1.67	4.18	5.70	1	1
B2	7	5	5.21	0.67	5.70	2	2
B3	6	4	5.52	1.05	4.53	2	2
C1	1	2	7.49	6.44	1.58	3	3
C2	4	9	0.33	5.55	6.04	1	1

$$d(p_1, p_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Like, Share and Subscribe to Mahesh Huddar — Visit: vtupulse.com

K = 3

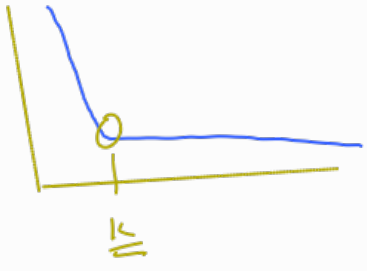
SSE $A_1 \rightarrow (1.94)^2 + (1.67)^2 + (0.33)^2 = 6.66$

$B_1 \rightarrow (1.05)^2 + (1.05)^2 + (0.67)^2 = 2.65$

$C_1 \rightarrow (1.58)^2 + (1.58)^2 = 5$

14.31 graatola

Chosing k-val



if graphis not as a shape of elbow then pick k value according to your preference

k-means

Strength	Weakness
efficient, simple, fast handles variety of data.	→ Struggles if data groups have diff sizes, shapes, densities, → Unable to effectively handle outliers, due to large distances ↑SSE. if shape is not global/circular. → not k must be known beforehand → Sensitive to initial condition Algo may trapped in Local optimum. (Bad grouping)

↳ k-mean Assumes all data groups as of same size, shape, density.

Hierarchial clustering:: → Biggest advantage
↓
no need to know the no. of clusters in advance

Agglomerative → goes point by point and merges two points into 1 clusters that are closest

Decisive:: goes through dataset and looks for points that are farthest apart and forms seperate clusters.

Cluster distance Measures in Allogromative::

Single link :: absolute closet

complete link :: absolute farthest

Avg link:: Avg dis b/w all points in cluster A and cluster B.

↓

effects the tree

Step by step: Algromative Algo

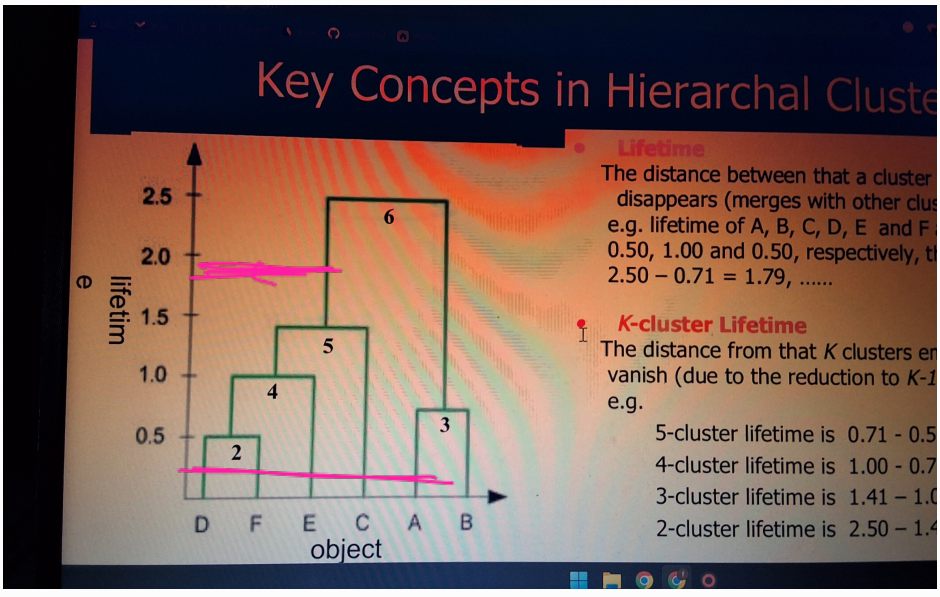
1) Make a distance Matrix (How far every point is and find min dis.

2) every point starts alone, as its own clucker.

3) Now keep on repeating until 1 cluster via Cluster Distance we achieve ↓ that contains all object.

lifetime in dendrogram → How long a cluster survives Before it merged with a group

long life time means → group is highly stable



(A,B) → 0.71
after merging
2.5
lifetime → 2.5 - 0.71
of (A,B) → 1.79

(K-cluster lifetime) → look at the longest vertical line

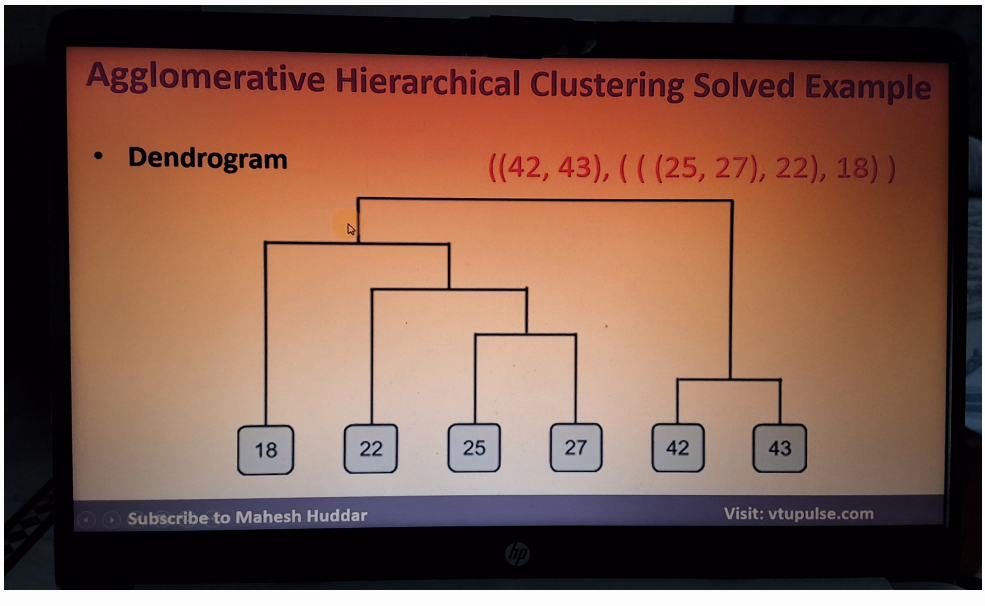
weakness of Agglomerative

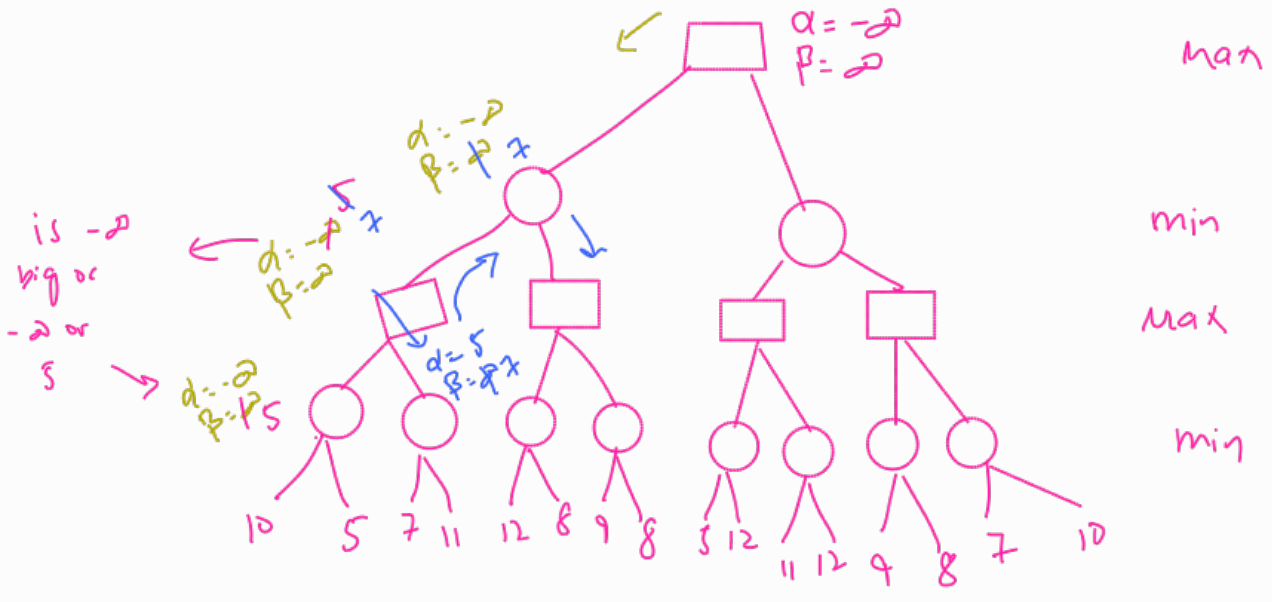
1) No undo button, once 2 points have been merged they have Bee merged

2) Highly sensitive to outliers / using different cluster distance measures changes your tree
e.g min / Avg / max.

3) Very slow

[42,43], 18 → (42,18), (43, 18) → (24, 25)
→ 24





α ≥ β pruning

max → update α
min → update β